

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



ĐỀ TÀI: HƯỚNG TIẾP CẬN MỚI CHO HỆ THỐNG TƯ VẤN MUA BÁN XE

Học phần: **ĐỒ ÁN TỐT NGHIỆP**

Giảng viên hướng dẫn: **PGS.TS NGUYỄN THANH BÌNH**

Thành viên nhóm:

Nguyễn Văn Trung Chính - 22280007

Nguyễn Xuân Việt Đức - 22280012

Bành Đức Khánh - 22280044

Lê Trọng Nghĩa - 22280059

TP. Hồ Chí Minh, Việt Nam

Tháng 7, 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan đồ án tốt nghiệp với đề tài **“Hướng tiếp cận mới cho hệ thống tư vấn mua bán xe”** là công trình nghiên cứu của nhóm chúng em dưới sự hướng dẫn của **PGS.TS Nguyễn Thanh Bình**.

Các số liệu, kết quả trình bày trong đồ án là trung thực và chưa từng được ai công bố trong bất kỳ công trình nào khác. Tất cả các nguồn tài liệu tham khảo đều được trích dẫn đầy đủ và rõ ràng.

Nhóm chúng em xin chịu hoàn toàn trách nhiệm về nội dung của đồ án này.

TP. Hồ Chí Minh, tháng 7, 2026

Nhóm sinh viên thực hiện

Nguyễn Văn Trung Chính

Nguyễn Xuân Việt Đức

Bành Đức Khánh

Lê Trọng Nghĩa

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Tên đề tài: Hướng tiếp cận mới cho hệ thống tư vấn mua bán xe

Sinh viên thực hiện:

- Nguyễn Văn Trung Chính – 22280007
- Nguyễn Xuân Việt Đức – 22280012
- Bành Đức Khánh – 22280044
- Lê Trọng Nghĩa – 22280059

Giảng viên hướng dẫn: PGS.TS Nguyễn Thanh Bình

Nhận xét:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm đánh giá:

TP. Hồ Chí Minh, ngày tháng năm 2026

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

PGS.TS Nguyễn Thanh Bình

NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

Tên đề tài: Hướng tiếp cận mới cho hệ thống tư vấn mua bán xe

Sinh viên thực hiện:

- Nguyễn Văn Trung Chính – 22280007
- Nguyễn Xuân Việt Đức – 22280012
- Bành Đức Khánh – 22280044
- Lê Trọng Nghĩa – 22280059

Giảng viên phản biện:

Nhận xét:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm đánh giá:

TP. Hồ Chí Minh, ngày tháng năm 2026

Giảng viên phản biện

(Ký và ghi rõ họ tên)

THUYẾT MINH CHỈNH SỬA ĐỀ TÀI

STT	Nội dung góp ý	Nội dung chỉnh sửa	Trang
1			
2			
3			
4			
5			

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

Nhóm sinh viên thực hiện

(Ký và ghi rõ họ tên)

PGS.TS Nguyễn Thanh Bình

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án tốt nghiệp, nhóm em đã nhận được sự hướng dẫn, hỗ trợ và tạo điều kiện từ quý thầy cô hướng dẫn và nhà trường. Bằng tất cả sự trân trọng, nhóm em xin gửi lời cảm ơn sâu sắc đến quý thầy cô hướng dẫn và nhà trường đã đồng hành cùng nhóm em trong quá trình thực hiện đề tài.

Lời đầu tiên, em xin bày tỏ lòng biết ơn đến thầy PGS.TS Nguyễn Thanh Bình, giảng viên hướng dẫn, người đã nhiệt tình định hướng, góp ý và hỗ trợ nhóm em trong suốt thời gian nghiên cứu và hoàn thiện khóa luận. Những nhận xét và sự hỗ trợ tận tâm của thầy là nền tảng quan trọng giúp chúng em hoàn thành tốt đề tài.

Chúng em cũng xin gửi lời cảm ơn đến Khoa Toán - Tin – Trường Đại học Khoa Học Tự Nhiên TP. Hồ Chí Minh đã tạo môi trường học tập thuận lợi, cung cấp kiến thức chuyên môn và kỹ năng cần thiết trong suốt quá trình học tập tại trường.

Và, em xin cảm ơn các thầy và các thầy/cô bộ môn đã truyền đạt những kiến thức bổ ích và luôn tận tâm trong công tác giảng dạy, giúp chúng em có được nền tảng vững chắc để thực hiện đề tài này.

Chúng em đã cố gắng hoàn thành đề tài, song kết quả cuối cùng có thể khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý của quý thầy cô để hoàn thiện hơn trong tương lai.

TP. Hồ Chí Minh, tháng 7, 2026
Chúng em trân trọng cảm ơn thầy !

Mục lục

Lời cam đoan	i
Nhận xét của giảng viên hướng dẫn	ii
Nhận xét của giảng viên phản biện	iii
Thuyết minh chỉnh sửa đề tài	iv
Lời cảm ơn	v
1 GIỚI THIỆU	1
1.1 Sơ lược về dự án	1
1.2 Kiến trúc hệ thống tổng quát	1
1.3 Giới thiệu về nguồn dữ liệu	2
1.3.1 Tầng Bronze (Dữ liệu thô)	2
1.3.2 Tầng Silver (Dữ liệu chuẩn hóa)	3
1.3.3 Tầng Gold (Dữ liệu phục vụ ứng dụng)	3
1.4 Ý nghĩa và Thách thức	3
2 CƠ SỞ LÝ THUYẾT	5
2.1 Kiến trúc Dữ liệu và Điều phối (Data Engineering & Orchestration)	5
2.2 Hệ thống đề xuất (Recommendation System)	5
2.3 Tìm kiếm ngữ nghĩa và Truy xuất kết hợp (Hybrid Search)	5
2.4 Mô hình Retrieval-Augmented Generation (RAG) và Kiến trúc Agentic	6
3 PHƯƠNG PHÁP ĐỀ XUẤT	8
3.1 Thu thập và xử lý dữ liệu	8
3.1.1 Xác định thông tin cần lưu trữ	8
3.1.2 Tiến hành thu thập dữ liệu	8
3.1.2.1 Vượt qua cơ chế chống bot (Cloudflare Turnstile)	8
3.1.2.2 Kiến trúc crawler	9
3.1.2.3 Điều phối tự động bằng Temporal	9
3.1.3 Xử lý và chuyển đổi dữ liệu (dbt Pipeline)	10
3.1.3.1 Nạp dữ liệu vào tầng Bronze (load_bronze)	10
3.1.3.2 Chuyển đổi dữ liệu bằng dbt (Medallion Architecture)	10
3.1.3.3 Refresh materialized view và tính toán ML	11
3.1.3.4 Tổng kết luồng xử lý dữ liệu	11

3.2	Kiến trúc hệ thống, backend và hệ thống đề xuất	12
3.2.1	Giới thiệu	12
3.2.2	Kiến trúc tổng quan	12
3.2.3	Kiến trúc API	14
3.2.4	Thiết kế Database	15
3.2.5	Data Pipeline	16
3.2.6	Hệ thống Authentication	16
3.2.7	Tối ưu hiệu năng	17
3.2.7.1	Cache trong tiến trình	17
3.2.7.2	Tối ưu truy vấn Database	17
3.2.8	Hệ thống Recommendation	18
3.2.8.1	Stage 1 — Candidate Generation	18
3.2.8.2	Stage 2 — Ranking	19
3.2.8.3	Stage 3 — Re-ranking	19
3.2.8.4	Tóm tắt luồng xử lý	20
3.3	Xây dựng chatbot tư vấn	20
3.3.1	Tổng quan và Đặt vấn đề	20
3.3.1.1	Đặt vấn đề	20
3.3.1.2	Phạm vi tư vấn của hệ thống	20
3.3.2	Cơ sở lý thuyết và Công nghệ áp dụng	21
3.3.2.1	Hạn chế của mô hình RAG truyền thống (Naive RAG)	21
3.3.2.2	Tổng quan về Agentic RAG	22
3.3.2.3	LangGraph	23
3.3.3	Kiến trúc tổng thể	23
3.3.3.1	Sơ đồ kiến trúc tổng thể của hệ thống Agentic RAG	23
3.3.3.2	Quy trình thu thập và xử lý dữ liệu (Data Ingestion)	24
3.3.3.3	Quy trình tương tác với người dùng và Luồng xử lý hội thoại	25
3.3.4	Thiết kế luồng Agentic RAG bằng LangGraph	26
3.3.4.1	Khởi tạo tài nguyên và Chu trình thực thi (Execution Cycle)	26
3.3.4.2	Định nghĩa Trạng thái Hội thoại (AgenticState)	27
3.3.4.3	Cơ chế quản lý hồ sơ khách hàng và Thu thập thông tin (Slot-filling)	28
3.3.4.4	Kiến trúc Sơ đồ Đồ thị LangGraph	29
3.3.4.5	So sánh với thiết kế SELF-RAG / Adaptive-RAG	31
3.3.5	Đánh giá mô hình (Evaluation)	32
3.3.5.1	Mục tiêu và Phương pháp luận	32
3.3.5.2	Quy trình Đánh giá	32
3.3.5.3	Xây dựng Bộ dữ liệu kiểm thử (Test Dataset)	33
3.3.5.4	Hệ thống Tiêu chí Đánh giá GEval theo Luồng Nghiệp vụ	33
3.3.5.5	Kết quả đánh giá	33

3.3.6	Kết luận và Hướng phát triển	35
3.3.6.1	Kết quả đạt được	35
3.3.6.2	Hạn chế còn tồn tại	36
3.3.6.3	Định hướng phát triển	36
3.4	Mô hình định giá xe	36
3.4.1	Giới thiệu	36
3.4.2	Dữ liệu và tiền xử lý	36
3.4.3	Xây dựng đặc trưng	36
3.4.4	Huấn luyện và so sánh các hướng tiếp cận	38
3.4.5	Triển khai	38
4	KẾT QUẢ ĐẠT ĐƯỢC VÀ ĐÁNH GIÁ	39
4.1	Kết quả và Đánh giá	39
4.1.1	Kết quả xây dựng Giao diện hệ thống	39
4.1.2	Đánh giá Hệ thống Chatbot (Qualitative Evaluation)	39
4.1.3	Khả năng truy xuất dữ liệu (Hybrid Retrieval)	40
4.1.4	Mô hình định giá xe (Price Estimation)	40
4.1.5	Tổng kết	40
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	41
5.1	Kết Luận	41
5.2	Ứng Dụng Thực Tế	41
5.3	Hướng Phát Triển	42

Danh sách hình vẽ

1.1	<i>Cars.com</i> – Trang web chính được sử dụng để thu thập dữ liệu.	1
1.2	Kiến trúc tổng quát của hệ thống tư vấn mua bán xe.	2
2.1	Mô hình RAG tổng quát làm nền tảng lý thuyết cho hệ thống chatbot tư vấn xe .	6
3.1	Kiến trúc tổng quan hệ thống	13
3.2	Kiến trúc phân tầng của Backend API	14
3.3	Kiến trúc multi-schema database	15
3.4	Luồng hoạt động của Recommendation Engine	18
3.5	Luồng xử lý RAG truyền thống (Naive RAG)	22
3.6	Kiến trúc Agentic RAG trong hệ thống tư vấn mua bán xe	24
3.7	Sơ đồ luồng xử lý và định tuyến rẽ nhánh trong mạng đồ thị LangGraph	30
3.8	Biểu đồ phân phối điểm số đánh giá theo luồng nghiệp vụ (Ngưỡng đạt: 0.7) . .	34
3.9	Điểm số đánh giá chi tiết theo từng tiêu chí GEval	35
3.10	Kiến trúc pipeline của mô hình định giá xe	37

Danh sách bảng

3.1	Tóm tắt các bước của pipeline dữ liệu từ thu thập đến phục vụ	12
3.2	Bảy đặc trưng và trọng số của WeightedLinearRanker	19
3.3	Tóm tắt ba giai đoạn của Recommendation Engine	20
3.4	Cấu trúc biến trạng thái AgenticState trong LangGraph	28
3.5	Đặc tả chức năng các nút tác vụ (Nodes) trong đồ thị LangGraph	31
3.6	Phân bổ tập dữ liệu kiểm thử và cơ sở ánh xạ luồng nghiệp vụ	33
3.7	Đặc tả các bộ tiêu chí chấm điểm và tham số đầu vào của GEval	33
3.8	Kết quả đánh giá DeepEval GEval theo từng luồng nghiệp vụ (Thang điểm 0–1)	34
3.9	So sánh các hướng tiếp cận định giá trên tập kiểm thử	38

1 GIỚI THIỆU

1.1 Sơ lược về dự án

Trong bối cảnh kỷ nguyên số và xu hướng chuyển đổi số diễn ra mạnh mẽ, mua sắm qua các nền tảng trực tuyến đã trở thành thói quen phổ biến của người dùng tại Việt Nam cũng như trên toàn thế giới. Cùng với đó, thị trường mua bán ô tô đang phát triển sôi động, với số lượng mẫu xe đa dạng và mức giá biến động theo nhiều yếu tố như thương hiệu, phiên bản, tình trạng xe và tính năng đi kèm. Sự phong phú về lựa chọn giúp người mua có nhiều cơ hội tiếp cận sản phẩm phù hợp, nhưng cũng làm tăng khó khăn trong việc so sánh, đánh giá và xác định phương án tối ưu theo nhu cầu và ngân sách.

Xuất phát từ thực tế đó, nhóm đề xuất phát triển một hệ thống tư vấn và gợi ý xe nằm trên hạ tầng đám mây với tính sẵn sàng cao dựa trên dữ liệu thu thập từ website bán xe **Cars.com**, bao gồm nhiều mức giá và tình trạng xe khác nhau. Hệ thống tập trung vào việc tổng hợp thông tin, chuẩn hóa dữ liệu và hỗ trợ người dùng sàng lọc theo các tiêu chí quan trọng như khoảng giá, dòng xe, mục đích sử dụng và một số thông số kỹ thuật cơ bản. Từ đó, hệ thống hiển thị danh sách lựa chọn phù hợp hơn, giúp người dùng nhanh chóng khoanh vùng các mẫu xe đáp ứng đúng nhu cầu.

Với bộ dữ liệu hơn **5.300 xe** (VIN duy nhất) đã thu thập và cập nhật hàng tuần, dự án không chỉ dừng lại ở việc gợi ý lựa chọn hiện tại mà còn hướng tới việc mở rộng trong việc phân tích xu hướng thị trường trong tương lai. Cụ thể, từ biến động giá theo thời gian và mức độ quan tâm của người mua, hệ thống có thể hỗ trợ nhận định xu hướng tăng hoặc giảm giá và tình hình cung cầu của một số dòng xe. Bên cạnh đó, việc thu thập và khai thác các nhận xét của khách hàng cũng tạo cơ hội cho việc phân tích cảm xúc và nhu cầu người dùng đối với từng dòng xe, qua đó bổ sung thêm một lớp thông tin hữu ích cho quá trình ra quyết định.

Về giải pháp kỹ thuật, dự án phát triển hệ thống theo ba hướng chính:

- **Recommender System (Hệ thống đề xuất):** Hệ thống đề xuất lai (hybrid) 3 giai đoạn bao gồm: (1) sinh ứng viên từ 4 nguồn (Collaborative Filtering qua bảng item_similarity, Content-based, Vector search qua Qdrant, Popularity), (2) xếp hạng bằng bộ chấm điểm tuyến tính 7 đặc trưng, (3) tái xếp hạng bằng MMR để đảm bảo đa dạng kết quả.
- **Agentic Chatbot (Chatbot tư vấn thông minh):** Xây dựng chatbot dạng agent sử dụng **LangGraph** với luồng hội thoại đa nhánh (7 loại intent: compare, analytics, specs, specific, vague, chitchat, off_topic). Chatbot kết hợp truy vấn SQL trực tiếp trên gold.vehicles và tìm kiếm ngữ nghĩa qua Qdrant để đưa ra tư vấn cá nhân hóa, có dẫn chứng dữ liệu thực.
- **Data Pipeline tự động:** Hệ thống pipeline dữ liệu tự động chạy hàng tuần sử dụng **Temporal** để điều phối toàn bộ luồng: thu thập dữ liệu (crawl) → chuyển đổi dữ liệu (dbt medallion) → tính toán ML (cosine similarity, embeddings).

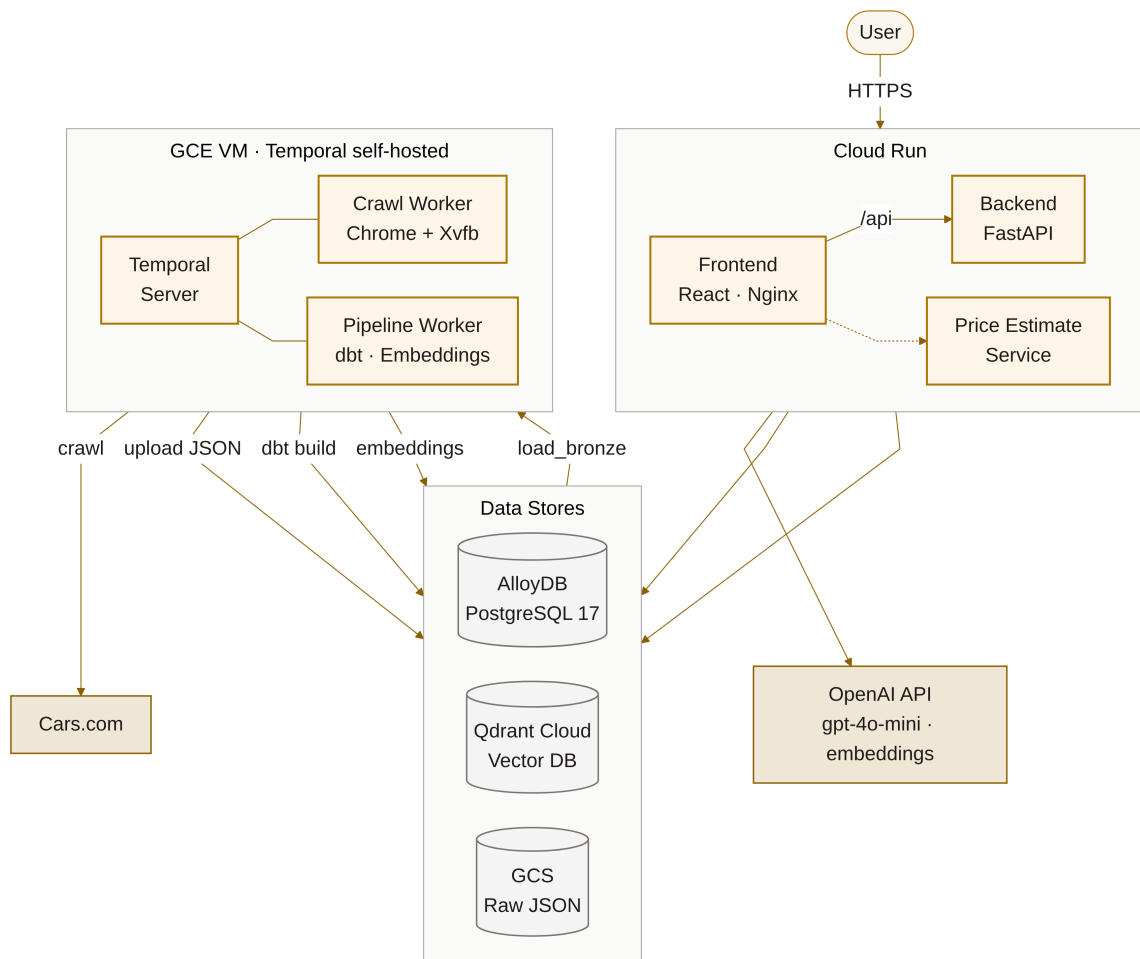


Hình 1.1: *Cars.com* – Trang web chính được sử dụng để thu thập dữ liệu.

1.2 Kiến trúc hệ thống tổng quát

Hệ thống được xây dựng trên nền tảng đám mây Google Cloud Platform (GCP) với kiến trúc microservices, bao gồm các thành phần chính như sau:

- **Frontend:** Ứng dụng web đơn trang (SPA) xây dựng bằng React + TypeScript + Tailwind CSS, triển khai trên Google Cloud Run dưới dạng container nginx.
- **Backend:** API server xây dựng bằng FastAPI (Python), triển khai trên Google Cloud Run, cung cấp các endpoint cho tìm kiếm, đề xuất, chat, xác thực và quản lý tương tác người dùng.
- **Database:** AlloyDB (PostgreSQL 17) lưu trữ dữ liệu theo kiến trúc medallion (bronze/silver/gold). Qdrant Cloud làm vector database cho tìm kiếm ngữ nghĩa.
- **Data Pipeline:** Temporal server tự triển khai (self-hosted) trên GCE VM điều phối toàn bộ luồng thu thập và xử lý dữ liệu tự động hàng tuần.
- **Chuyển đổi dữ liệu:** dbt (data build tool) thực hiện chuyển đổi dữ liệu từ bronze thô sang silver (chuẩn hóa theo mô hình chiều) và gold (bảng phục vụ ứng dụng).



Hình 1.2: Kiến trúc tổng quát của hệ thống tư vấn mua bán xe.

1.3 Giới thiệu về nguồn dữ liệu

Bộ dữ liệu được tổ chức theo **kiến trúc medallion** (Bronze → Silver → Gold) trên cơ sở dữ liệu PostgreSQL (AlloyDB), với dữ liệu được chuyển đổi tự động bằng dbt.

1.3.1 Tầng Bronze (Dữ liệu thô)

Sau khi thu thập dữ liệu trên trang web Cars.com, dữ liệu thô được lưu trữ trên **Google Cloud Storage (GCS)** theo cấu trúc phân vùng ngày:

- Mỗi file JSON chứa toàn bộ thông tin của một chiếc xe (thông tin bài đăng, thông số kỹ thuật, tính năng, hình ảnh, thông tin người bán, đánh giá).
- Cấu trúc lưu trữ: `gs://incremental_raw/dt=YYYY-MM-DD/<page>/<vin>.json`
- Hình ảnh xe và người bán được lưu kèm theo trong thư mục con.

Dữ liệu từ GCS sau đó được nạp vào bảng `bronze.raw_listings` trên PostgreSQL dưới dạng JSONB. Bảng này là **append-only** — mỗi file JSON tương ứng một hàng, khóa idempotent là `file_hash` (SHA-256 của nội dung file). Việc chạy lại pipeline không tạo bản ghi trùng lặp.

1.3.2 Tầng Silver (Dữ liệu chuẩn hóa)

Tầng Silver do **dbt** tự động xây dựng từ tầng Bronze, bao gồm hai bước:

1. **Staging:** Giải trùng lặp theo VIN (lấy bản ghi mới nhất theo `crawl_date`), sau đó trích xuất và phẳng hóa (flatten) cấu trúc JSON lồng nhau thành các trường riêng biệt.
2. **Mô hình chiều (Dimensional model):** Tổ chức thành bảng fact và dimension:
 - `fct_listing`: Bảng fact chính — mỗi hàng là một bài đăng xe (giá, quãng đường, tình trạng, động cơ, v.v.)
 - `dim_car_model`: Thông tin hãng xe và dòng xe
 - `dim_seller`: Thông tin người bán/đại lý
 - `dim_feature`: Danh mục tính năng xe
 - `dim_listing_image`: Hình ảnh xe
 - `bridge_listing_feature`: Liên kết N-N giữa bài đăng và tính năng
 - `fct_model_rating` và `fct_model_review`: Đánh giá và nhận xét theo dòng xe

1.3.3 Tầng Gold (Dữ liệu phục vụ ứng dụng)

Tầng Gold cũng do dbt xây dựng, là các bảng denormalized tối ưu cho truy vấn từ backend. Các bảng chính:

- `gold.vehicles`: Bảng chính (merge by VIN, incremental) — một hàng mỗi xe, chứa tất cả thông tin cần thiết (giá, hãng, dòng, tính năng, hình ảnh chính, đánh giá mô hình). Hiện tại có **5.337 VIN** duy nhất.
- `gold.vehicle_price_history`: Lịch sử giá xe theo ngày thu thập.
- `gold.car_models`: Thông tin dòng xe tổng hợp (đánh giá, tỷ lệ khuyến nghị).
- `gold.sellers`: Thông tin đại lý/người bán.
- `gold.reviews`: Nhận xét chi tiết từ người dùng.
- `gold.vehicle_features` và `gold.vehicle_images`: Tính năng và hình ảnh xe.

Ngoài ra, tầng Gold còn bao gồm các **bảng user-domain** do ứng dụng FastAPI ghi trực tiếp (không qua dbt):

- `gold.users`: Thông tin người dùng (đăng ký, xác thực).
- `gold.user_interactions`: Lịch sử tương tác (view, click, compare, save, favorite, contact, inquiry).
- `gold.item_similarity`: Ma trận tương đồng xe (tính bằng cosine similarity, cập nhật hàng tuần).
- `gold.chat_sessions` và `gold.chat_messages`: Lịch sử hội thoại chatbot của người dùng đã đăng nhập.

1.4 Ý nghĩa và Thách thức

Bộ dữ liệu thu thập từ *Cars.com* mang lại một số **ưu điểm** đáng chú ý:

- **Tính cập nhật cao:** Dữ liệu được thu thập tự động hàng tuần qua pipeline Temporal, đảm bảo thông tin bài đăng và giá bán phản ánh tương đối sát thị trường tại thời điểm hiện tại.
- **Thông tin đa dạng:** Dữ liệu bao gồm nhiều khía cạnh như thông tin cơ bản của xe, các tính năng (feature), hình ảnh, thông tin người bán và đánh giá từ người dùng.

- **Nguồn dữ liệu phổ biến:** Website có lượng người dùng lớn, giúp thu thập được nhiều nhận xét thực tế, hỗ trợ phân tích hành vi và nhu cầu của người mua.
- **Kiến trúc dữ liệu chuẩn:** Áp dụng kiến trúc medallion (Bronze → Silver → Gold) với dbt, đảm bảo dữ liệu được chuẩn hóa, có thể truy vết nguồn gốc (lineage) và dễ mở rộng.
- **Idempotent và incremental:** Pipeline thu thập và xử lý dữ liệu đảm bảo tính idempotent — chạy lại bất kỳ bước nào cũng không tạo dữ liệu trùng lặp, nhờ cơ chế dedup theo `file_hash` (bronze) và merge theo VIN (gold).

Tuy nhiên, bộ dữ liệu cũng tồn tại một số **thách thức**:

- **Chống bot (Anti-scraping):** Cars.com sử dụng hệ thống bảo vệ Cloudflare Turnstile, khiến việc thu thập dữ liệu bằng trình duyệt headless thông thường thất bại. Nhóm phải sử dụng SeleniumBase UC mode (Undetected ChromeDriver) kết hợp Xvfb trên máy thật để vượt qua cơ chế này. Hệ quả là crawler **không thể chạy trong Docker container** mà phải chạy trực tiếp trên máy host.
- **Về mặt địa lý:** Dữ liệu chủ yếu phản ánh thị trường ô tô tại Mỹ, do đó giá cả có thể lệch so với các quốc gia khác một số thời điểm.
- **Về mặt thời gian:** Giá và tình trạng xe thay đổi nhanh theo thời điểm. Mặc dù hệ thống đã có cơ chế thu thập hàng tuần, nhưng vẫn tồn tại độ trễ nhất định.
- **Về chất lượng dữ liệu:** Một số bài đăng có thể thiếu thông tin hoặc trình bày không đầy đủ; đồng thời người bán có thể không công bố (hoặc mô tả mơ hồ) các khuyết điểm của xe, gây khó khăn trong việc đối soát và đánh giá chính xác tình trạng xe thực tế.

Nhìn chung, bộ dữ liệu này là nền tảng quan trọng để phát triển các chức năng sàng lọc, gợi ý xe theo nhu cầu và xây dựng chatbot tư vấn dựa trên hệ thống đề xuất lai (hybrid Recommender System) và chatbot thông minh dạng agent (Agentic LangGraph chatbot).

2 CƠ SỞ LÝ THUYẾT

2.1 Kiến trúc Dữ liệu và Điều phối (Data Engineering & Orchestration)

Trong các hệ thống phân tích và ứng dụng dữ liệu hiện đại, việc quản lý vòng đời dữ liệu từ thô đến tinh là yếu tố cốt lõi. Hệ thống áp dụng **Kiến trúc Medallion (Medallion Architecture)** - một mẫu thiết kế tổ chức dữ liệu thành ba tầng logic:

- **Tầng Bronze (Dữ liệu thô):** Lưu trữ dữ liệu nguyên bản từ các nguồn ngoại vi (ví dụ: kết quả crawl từ website). Dữ liệu thường có dạng bán cấu trúc (JSON) và được lưu trữ theo cơ chế *append-only*, đảm bảo khả năng truy vết (lineage) và không bị mất mát thông tin.
- **Tầng Silver (Dữ liệu chuẩn hóa):** Dữ liệu từ tầng Bronze được làm sạch, giải trùng lặp (deduplication) và cấu trúc hóa (flattening) thành các bảng theo mô hình chiều (Dimensional Modeling) gồm fact và dimension.
- **Tầng Gold (Dữ liệu phục vụ ứng dụng):** Dữ liệu được tổng hợp và phi chuẩn hóa (denormalized) thành các bảng tối ưu cho truy vấn nghiệp vụ (ví dụ: dashboard, API backend, hoặc huấn luyện mô hình Machine Learning).

Để tự động hóa luồng chuyển đổi qua các tầng, hệ thống ứng dụng lý thuyết **Điều phối Workflow (Workflow Orchestration)** thông qua công cụ như Temporal hoặc Airflow. Dữ liệu được xử lý theo các bước tuần tự (DAG), đồng thời tuân thủ nguyên tắc **Idempotent** (chạy lại một tác vụ nhiều lần không làm thay đổi trạng thái dữ liệu mong muốn) để đảm bảo tính an toàn (fail-safe).

2.2 Hệ thống đề xuất (Recommendation System)

Hệ thống đề xuất dự đoán "đánh giá" hoặc "sự ưa thích" của người dùng đối với một mặt hàng. Trong thực tế, các hệ thống quy mô lớn thường áp dụng kiến trúc **Đề xuất Đa giai đoạn (Multi-stage Recommendation)**:

1. **Sinh ứng viên (Candidate Generation / Recall):** Trích xuất một tập nhỏ các mặt hàng tiềm năng từ cơ sở dữ liệu lớn (hàng triệu items) bằng nhiều thuật toán chạy song song:
 - *Lọc cộng tác (Collaborative Filtering):* Dựa trên hành vi quá khứ của những người dùng có sở thích tương tự.
 - *Lọc dựa trên nội dung (Content-based):* Dựa trên sự tương đồng về đặc tính vật lý (hãng, giá, thông số).
 - *Tìm kiếm theo vector (Vector / Semantic):* Truy xuất các mặt hàng tương đồng về ngữ nghĩa trong không gian embedding thông qua vector database.
 - *Phổ biến (Popularity):* Đề xuất các mặt hàng được tương tác nhiều nhất (giải quyết bài toán cold-start).
2. **Xếp hạng (Ranking):** Sử dụng một mô hình đánh giá (như *Weighted Linear Ranker*) để tính điểm (score) cho từng ứng viên dựa trên tập hợp các đặc trưng (features) kết hợp.
3. **Tái xếp hạng (Re-ranking):** Áp dụng thuật toán **MMR (Maximal Marginal Relevance)** nhằm giải quyết vấn đề danh sách đề xuất quá nhàm chán (ví dụ: toàn bộ xe cùng một hãng). MMR cân bằng giữa độ chính xác (*Relevance*) và độ đa dạng (*Diversity*) bằng công thức phạt sự trùng lặp với những mặt hàng đã được chọn trước đó.

2.3 Tìm kiếm ngữ nghĩa và Truy xuất kết hợp (Hybrid Search)

Tìm kiếm ngữ nghĩa (Semantic Search) vượt qua giới hạn của tìm kiếm từ khóa (keyword matching) bằng cách hiểu ngữ cảnh truy vấn. Văn bản được chuyển đổi thành **Vector nhúng (Embeddings)** trong không gian đa chiều

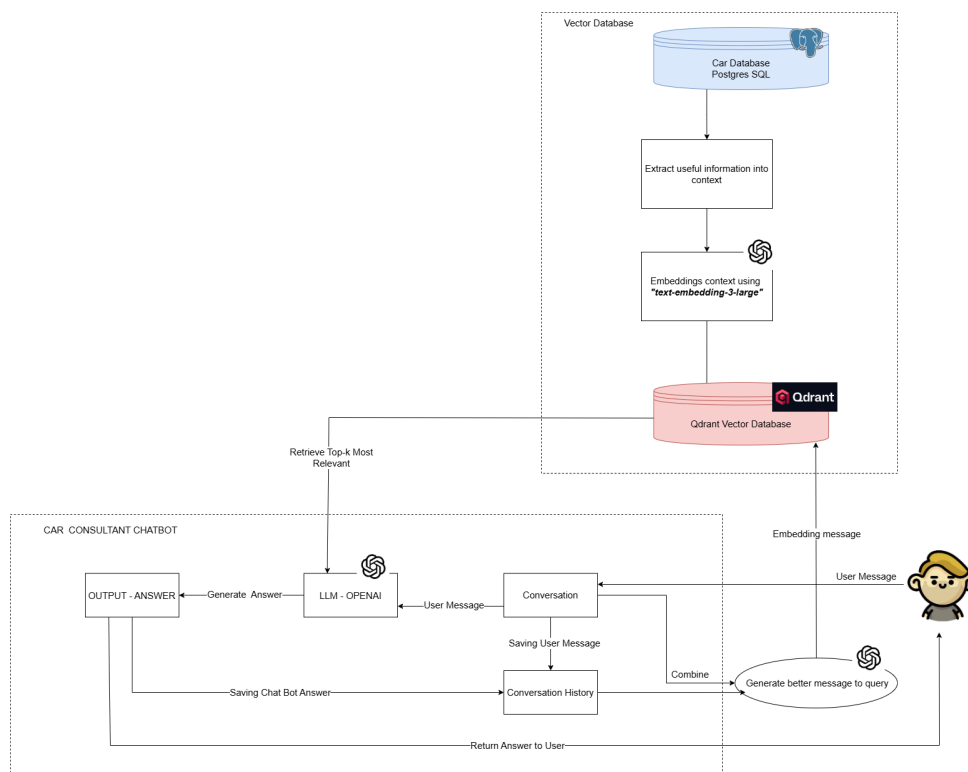
nhờ mô hình học sâu. Mức độ tương đồng giữa truy vấn và tài liệu tỷ lệ nghịch với khoảng cách vector (Cosine, Euclidean).

Cơ sở dữ liệu Vector (Vector Database) như Qdrant sử dụng các thuật toán *Approximate Nearest Neighbor (ANN)* để truy vấn vector tốc độ cao. Tuy nhiên, tìm kiếm ngữ nghĩa đơn thuần có thể bỏ qua các ràng buộc chính xác (ví dụ: "giá dưới 500 triệu"). Do đó, **Tìm kiếm lai (Hybrid Search)** được áp dụng:

- **Lọc cứng (Hard-filter):** Dùng truy vấn SQL truyền thống (RDBMS) để lọc tuyệt đối các điều kiện như giá cả, hãng xe, năm sản xuất.
- **Kết hợp (Fusion):** Kết quả từ Vector Database và RDBMS được hòa trộn thành một ngữ cảnh thống nhất. Về mặt lý thuyết, một phương pháp phổ biến để hợp nhất nhiều danh sách xếp hạng là **Reciprocal Rank Fusion (RRF)** — xếp hạng lại các mặt hàng dựa trên nghịch đảo vị trí của chúng ở các danh sách kết quả độc lập. Trong hệ thống chatbot của dự án, phần truy xuất kết hợp hai nguồn (SQL hard-filter và Qdrant) theo hướng ghép ngữ cảnh (context concatenation) trước khi đưa vào mô hình sinh câu trả lời.

2.4 Mô hình Retrieval-Augmented Generation (RAG) và Kiến trúc Agentic

RAG (Retrieval-Augmented Generation) kết hợp khả năng **truy xuất thông tin (Retrieval)** từ cơ sở dữ liệu nội bộ và khả năng **sinh ngôn ngữ (Generation)** của LLM, giúp Chatbot trả lời dựa trên thông tin thực tế, tránh ảo giác (hallucination). Kiến trúc RAG tổng quát được minh họa trong Hình 2.1.



Hình 2.1: Mô hình RAG tổng quát làm nền tảng lý thuyết cho hệ thống chatbot tư vấn xe

Nâng cấp từ RAG tuyến tính, kiến trúc **Agentic RAG** định tuyến động (conditional routing) các câu hỏi dựa trên ý định (Intent). Mô hình được thiết kế dưới dạng **Đồ thị trạng thái (State Graph)** như *LangGraph*:

- **Quản lý trạng thái (State Management):** Mỗi phiên hội thoại duy trì một trạng thái lưu trữ ngữ cảnh, lịch sử và kết quả truy xuất.
- **Thu thập thông tin (Slot-filling):** Thay vì trả lời ngay, Agentic AI có khả năng nhận diện những thông tin còn thiếu (slots) trong yêu cầu (ví dụ: thiếu tầm giá). Chatbot sẽ chủ động đặt câu hỏi ngược lại người dùng

để lấp đầy các slots này trước khi thực hiện truy xuất dữ liệu, tạo ra luồng tư vấn cá nhân hóa và tự nhiên như con người.

3 PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Thu thập và xử lý dữ liệu

3.1.1 Xác định thông tin cần lưu trữ

Nhóm tiến hành khảo sát website *Cars.com* nhằm xác định cấu trúc hiển thị, phân loại nội dung và các trường thông tin có thể trích xuất. Kết quả khảo sát cho thấy dữ liệu trên website có cấu trúc tương đối thống nhất giữa các trang bài đăng, tạo điều kiện thuận lợi để tự động hóa quá trình thu thập và chuẩn hóa dữ liệu.

Nhằm mở rộng phạm vi dữ liệu, không chỉ giới hạn ở các thông số và nội dung bài đăng xe, nhóm tiếp tục thu thập thêm bằng cách điều hướng sang trang người bán (seller/dealer) và trang đánh giá xe (review). Cách tiếp cận này giúp bổ sung thông tin về đại lý, mức độ uy tín, cũng như các nhận xét thực tế từ người dùng. Dữ liệu sau đó được phân thành các nhóm chính như sau:

- **Thông tin bài đăng xe (Post):** trạng thái xe (mới hoặc cũ), tiêu đề, giá bán, số km đã đi, phương thức thanh toán trả góp, hình ảnh, và đường dẫn đến các trang chi tiết.
- **Thông tin chi tiết kỹ thuật (Vehicle Details):** Các thuộc tính cơ bản như hãng xe, dòng xe, năm sản xuất, loại động cơ, hộp số, nhiên liệu, tình trạng bảo hành, và mã VIN. Đây là nhóm thông tin quan trọng nhất, giúp định danh và phân loại xe.
- **Tính năng xe (Features):** Bao gồm danh sách các tính năng được chia theo nhóm như an toàn (Safety), tiện nghi (Convenience), công nghệ (Technology), hỗ trợ lái (Driver Assist), v.v.
- **Người bán (Seller/Dealer):** Thông tin đại lý hoặc cá nhân đăng bán gồm tên, địa chỉ, liên hệ, thời gian mở cửa, điểm đánh giá và mô tả.
- **Nhận xét của người dùng (Reviews/Ratings):** Thông tin phản hồi từ khách hàng như điểm đánh giá tổng quan, nội dung nhận xét, thời gian đánh giá, vị trí, và điểm chi tiết theo từng tiêu chí (Performance, Comfort, Interior, Reliability, Value, Exterior).

Sau khi khảo sát, nhóm nhận thấy mỗi xe sẽ mang một số VIN duy nhất nên đã quyết định lựa chọn mã **VIN** (Vehicle Identification Number) làm khóa định danh duy nhất cho mỗi bài đăng của xe, dùng để liên kết giữa các bảng dữ liệu trong giai đoạn xử lý và lưu trữ.

3.1.2 Tiến hành thu thập dữ liệu

3.1.2.1 Vượt qua cơ chế chống bot (Cloudflare Turnstile)

Thách thức lớn nhất trong quá trình thu thập dữ liệu là hệ thống bảo vệ **Cloudflare Turnstile** mà *Cars.com* sử dụng. Cơ chế này phát hiện và chặn các trình duyệt tự động (headless browser) thông thường, khiến các thư viện crawling phổ biến như Selenium headless hay Scrapy không thể truy cập nội dung trang.

Để giải quyết vấn đề này, nhóm sử dụng **SeleniumBase** ở chế độ **UC mode** (Undetected ChromeDriver) — một phương pháp ngắt kết nối ChromeDriver trong quá trình tải trang để Cloudflare không thể nhận diện (fingerprint) trình duyệt tự động. Khi phát hiện trang challenge “Just a moment...” hoặc “Verifying...”, hệ thống tự động gọi `uc_gui_click_captcha()` để giải captcha Turnstile.

Hệ quả quan trọng: crawler **phải chạy trên máy thật** (host) với trình duyệt Chrome đầy đủ và **Xvfb** (X Virtual Framebuffer) để mô phỏng màn hình ảo. Việc chạy trong Docker container là không khả thi vì Turnstile phát hiện môi trường container.

3.1.2.2 Kiến trúc crawler

Quá trình thu thập dữ liệu được xây dựng dưới dạng package Python (*crawler/*) với kiến trúc module hóa, chia thành ba giai đoạn chính:

3.1.2.2.1 Giai đoạn 1: Thu thập danh sách liên kết (*link_crawler.py*)

Module *link_crawler.py* sử dụng một phiên trình duyệt SeleniumBase duy nhất để duyệt các trang kết quả tìm kiếm trên *Cars.com*. Trình duyệt duy trì cookie Cloudflare xuyên suốt các trang để tránh bị challenge lặp lại. Mỗi trang kết quả chứa khoảng 20–25 liên kết bài đăng dạng */vehicledetail/...*

Danh sách URL được lưu theo từng trang trong thư mục *car_links/page_{n}.txt*. Cơ chế **resumable**: nếu file liên kết đã tồn tại và không rỗng, trang đó sẽ được bỏ qua. Giữa các trang có khoảng nghỉ ngẫu nhiên (2–3.5 giây) để tránh bị rate limit. Khi gặp lỗi, hệ thống tự động retry với backoff lũy thừa (tối đa 3 lần).

3.1.2.2.2 Giai đoạn 2: Thu thập chi tiết từng xe (*detail_scraper.py*)

Từ danh sách URL đã thu được, module *detail_scraper.py* mở nhiều phiên trình duyệt song song (mặc định tối đa 5 worker) để tăng tốc. Mỗi worker xử lý một phần danh sách URL và lưu kết quả ra file JSON riêng biệt.

Đối với mỗi URL xe, hệ thống thực hiện quy trình *scrape_full* gồm các bước:

1. **Scrape trang chi tiết xe (*scrape_listing*):** Tải HTML qua SeleniumBase UC mode, sau đó sử dụng **BeautifulSoup** để phân tích cú pháp (parse) và trích xuất ba nhóm thông tin qua các parser chuyên biệt:
 - *parse_post*: Trích xuất thông tin bài đăng (giá, trạng thái new/used, tiêu đề, quãng đường, thanh toán trả góp, hình ảnh, mã VIN).
 - *parse_seller*: Trích xuất thông tin người bán từ sidebar (tên đại lý, địa chỉ, đánh giá, liên kết dealer page).
 - *parse_car*: Trích xuất thông số kỹ thuật (động cơ, hộp số, nhiên liệu, drivetrain), danh sách tính năng theo nhóm (Safety, Comfort, Technology, ...), và thông tin review tổng hợp (điểm đánh giá, tỷ lệ khuyến nghị, nhận xét chi tiết theo tiêu chí).
2. **Scrape trang đại lý (*scrape_dealer*):** Nếu trang xe có liên kết đến trang dealer, hệ thống truy cập thêm để thu thập số điện thoại, giờ hoạt động, và thông tin bổ sung.
3. **Hợp nhất và lưu JSON:** Toàn bộ dữ liệu được hợp nhất thành một cấu trúc JSON duy nhất cho mỗi xe, lưu tại *raw_data/{page}/{index}.json*.

Cơ chế an toàn: mỗi file đã tồn tại sẽ được bỏ qua (skip), cho phép tiếp tục từ điểm dừng nếu quá trình bị gián đoạn. Mỗi URL thất bại sau 3 lần retry sẽ được ghi log nhưng không dừng pipeline.

3.1.2.2.3 Giai đoạn 3: Upload lên Google Cloud Storage (*gcs_uploader.py*)

Sau khi thu thập xong, module *gcs_uploader.py* đẩy toàn bộ file JSON và hình ảnh lên **Google Cloud Storage (GCS)** theo cấu trúc phân vùng ngày:

- JSON: *gs://incremental_raw/dt={crawl_date}/{page}/{file}.json*
- Hình ảnh: *gs://incremental_raw/dt={crawl_date}/images/{path}*

Mỗi lần chạy hàng tuần sử dụng một prefix *dt=* riêng biệt (ngày UTC hiện tại), đảm bảo **không bao giờ ghi đè** dữ liệu của các lần chạy trước. Cơ chế này hỗ trợ incremental loading ở tầng tiếp theo.

3.1.2.3 Điều phối tự động bằng Temporal

Toàn bộ ba giai đoạn trên được điều phối tự động bằng **Temporal** — một hệ thống workflow orchestration thay thế Airflow. Pipeline chạy theo lịch hàng tuần với workflow *WeeklyCrawlWorkflow*:

1. `crawl_links` → thu thập danh sách URL
2. `scrape_details` → thu thập chi tiết từng xe
3. `upload_gcs` → upload dữ liệu lên GCS

Temporal cung cấp cơ chế retry tự động với backoff policy riêng cho mỗi activity (ví dụ: scrape retry tối đa 2 lần vì tốn tài nguyên, upload retry tối đa 3 lần). Nếu một activity thất bại vượt quá giới hạn retry, workflow dừng lại (fail-stop) mà không ảnh hưởng đến dữ liệu đã thu thập thành công.

Crawler worker chạy trên **GCE VM** (cùng VM với Temporal server), sử dụng Chrome và Xvfb. Pipeline worker (dbt, embeddings) chạy trong Docker container trên cùng VM.

3.1.3 Xử lý và chuyển đổi dữ liệu (dbt Pipeline)

Sau khi dữ liệu thô được upload lên GCS, quá trình xử lý tiếp tục với hai bước: nạp dữ liệu vào database (load bronze) và chuyển đổi dữ liệu bằng dbt.

3.1.3.1 Nạp dữ liệu vào tầng Bronze (load_bronze)

Module `bronze.py` thực hiện nạp dữ liệu từ GCS vào bảng `bronze.raw_listings` trên PostgreSQL (AlloyDB):

1. **Scan GCS:** Liệt kê tất cả file `.json` trong prefix `dt={crawl_date}/` (hoặc toàn bộ bucket nếu chạy back-fill).
2. **Download và parse song song:** Sử dụng `ThreadPoolExecutor` (16 worker) để tải và phân tích JSON đồng thời. Mỗi file được tính SHA-256 hash làm khóa idempotent, đồng thời trích xuất các trường quan trọng từ payload (VIN, car_model_slug, crawled_at).
3. **Bulk insert:** Chèn hàng loạt vào bảng bronze (batch 500 hàng) với mệnh đề `ON CONFLICT (file_hash) DO NOTHING` — đảm bảo **idempotent**: chạy lại cùng dữ liệu không tạo bản ghi trùng.

3.1.3.2 Chuyển đổi dữ liệu bằng dbt (Medallion Architecture)

Sau khi dữ liệu đã nằm trong tầng bronze, **dbt** (data build tool) tự động chuyển đổi qua ba tầng:

3.1.3.2.1 Staging (Silver Staging)

Tầng staging giải quyết hai vấn đề chính:

- **Giải trùng lặp:** Model `stg_raw_latest` sử dụng window function `ROW_NUMBER() OVER (PARTITION BY vin ORDER BY crawl_date DESC)` để giữ lại bản ghi mới nhất cho mỗi VIN. Điều này quan trọng vì bronze là append-only — cùng một xe có thể được crawl nhiều lần.
- **Trích xuất và flatten JSON:** Các model staging (`stg_listings`, `stg_sellers`, `stg_car_models`, `stg_listing_features`, `stg_listing_images`, `stg_model_reviews`) trích xuất dữ liệu từ cột payload (JSONB) của bronze, phẳng hóa cấu trúc JSON lồng nhau thành các trường quan hệ riêng biệt. Ví dụ: `payload->'post'->'basic_desc'->'VIN'` được trích thành cột `vin`.

3.1.3.2.2 Silver (Mô hình chiều – Dimensional Model)

Tầng Silver tổ chức dữ liệu theo mô hình chiều (Dimensional Model) gồm các bảng fact và dimension:

- **Bảng Fact:**
 - `fact_listing`: Mỗi hàng là một bài đăng xe (incremental, delete+insert). Chứa khóa ngoại đến các bảng dimension, cùng các metric chính (giá, quãng đường, MPG, v.v.).
 - `fact_model_rating`: Điểm đánh giá tổng hợp theo dòng xe.
 - `fact_model_review`: Nhận xét chi tiết của từng người dùng.

- **Bảng Dimension:**

- `dim_car_model`: Hãng xe, dòng xe, slug.
- `dim_seller`: Tên đại lý, địa chỉ, thông tin liên hệ.
- `dim_feature`: Danh mục tính năng (tên, nhóm).
- `dim_listing_image`: Hình ảnh xe (URL, thứ tự).

- **Bảng Bridge:**

- `bridge_listing_feature`: Quan hệ nhiều-nhiều giữa bài đăng và tính năng.

3.1.3.2.3 Gold (Bảng phục vụ ứng dụng)

Tầng Gold là các bảng denormalized (phi chuẩn hóa có chủ đích) tối ưu cho truy vấn từ backend API:

- `gold.vehicles`: Bảng chính — merge by VIN (incremental strategy). Mỗi hàng chứa toàn bộ thông tin cần thiết cho frontend: giá, hãng, dòng, `body_type`, màu sắc, hình ảnh chính, số lượng tính năng, điểm đánh giá dòng xe. Trường `vehicle_id` là alias của `vin` để tương thích ngược với backend.
- `gold.vehicle_price_history`: Lịch sử biến động giá xe theo ngày crawl, cho phép theo dõi xu hướng giá.
- `gold.car_models`, `gold.sellers`, `gold.reviews`: Dữ liệu tổng hợp theo dòng xe, người bán, và nhận xét.
- `gold.vehicle_features`, `gold.vehicle_images`: Tính năng và hình ảnh chi tiết.

3.1.3.3 Refresh materialized view và tính toán ML

Toàn bộ pipeline được điều phối bởi `WeeklyPipelineWorkflow` — một workflow cha chuỗi (chain) ba workflow con theo thứ tự **fail-stop** (nếu bước trước thất bại thì không chạy bước sau):

1. `WeeklyCrawlWorkflow`: `crawl_links` → `scrape_details` → `upload_gcs` (chạy trên `CRAWL_TASK_QUEUE`, host machine).
2. `TransformWorkflow`: `ensure_partition` → `load_bronze` → `dbt_build` → `refresh_matviews` (chạy trên `PIPELINE_TASK_QUEUE`, Docker container).
3. `MLWorkflow`: `compute_item_similarity` || `embed_vehicles` || `embed_chatbot_vehicles` (chạy song song trên `PIPELINE_TASK_QUEUE`).

Sau khi dbt build hoàn tất, các bước ML thực hiện:

- `refresh_matviews`: Làm mới các materialized view phục vụ hệ thống đề xuất (`mv_popular_vehicles`, `mv_trending_models`).
- `compute_item_similarity`: Tính ma trận cosine similarity giữa các xe dựa trên đặc trưng (giá, quãng đường, đánh giá, v.v.), lưu vào `gold.item_similarity` phục vụ Collaborative Filtering.
- `embed_vehicles`: Tạo vector embedding cho từng xe bằng OpenAI `text-embedding-3-large` (3072 chiều), lưu vào Qdrant collection `car_chatbot_vectors` — phục vụ Vector Recall trong hệ thống đề xuất.
- `embed_chatbot_vehicles`: Chia nhỏ (chunk) thông tin xe thành các đoạn văn bản, tạo embedding và lưu vào Qdrant collection `car_vectorize` — phục vụ chatbot RAG.

Lưu ý quan trọng: Hai collection Qdrant phục vụ hai mục đích hoàn toàn khác nhau — nhằm lẫn giữa chúng sẽ phá vỡ hệ thống đề xuất hoặc chatbot. Ba bước `compute_item_similarity`, `embed_vehicles` và `embed_chatbot_vehicles` chạy **song song** (parallel) nhờ Temporal, giảm thời gian xử lý tổng thể.

3.1.3.4 Tổng kết luồng xử lý dữ liệu

Toàn bộ pipeline từ thu thập đến sẵn sàng phục vụ ứng dụng được tóm tắt như sau:

Bảng 3.1: Tóm tắt các bước của pipeline dữ liệu từ thu thập đến phục vụ

Bước	Mô tả	Công nghệ
1	Thu thập URL bài đăng	SeleniumBase UC mode
2	Thu thập chi tiết xe + dealer	SeleniumBase + BeautifulSoup
3	Upload dữ liệu thô lên GCS	Google Cloud Storage
4	Nạp vào bronze (JSONB)	Python + psycopg2
5	Staging: dedup + flatten JSON	dbt (SQL view)
6	Silver: mô hình chiều (dimensional)	dbt (SQL table)
7	Gold: bảng denormalized	dbt (incremental merge)
8	Refresh materialized views	SQL
9	Cosine similarity matrix	Python (scikit-learn)
10	Vector embeddings	OpenAI + Qdrant

Pipeline chạy hoàn toàn tự động hàng tuần qua Temporal WeeklyPipelineWorkflow, đảm bảo dữ liệu luôn được cập nhật mà không cần can thiệp thủ công.

3.2 Kiến trúc hệ thống, backend và hệ thống đề xuất

3.2.1 Giới thiệu

Hệ thống backend đóng vai trò là trung tâm xử lý chính của ứng dụng *Car Recommendation System*, chịu trách nhiệm tiếp nhận yêu cầu từ người dùng, xử lý nghiệp vụ, truy xuất dữ liệu và trả về kết quả gợi ý phù hợp. Backend hoạt động như một lớp trung gian, kết nối giữa giao diện người dùng (frontend) và các hệ thống lưu trữ dữ liệu phía sau, đảm bảo tính nhất quán, hiệu năng và khả năng mở rộng của toàn bộ hệ thống.

Backend được xây dựng dựa trên framework **FastAPI** với ngôn ngữ **Python**, cho phép phát triển các API theo kiến trúc RESTful với hiệu năng cao, khả năng mở rộng tốt và hỗ trợ tài liệu API tự động thông qua OpenAPI/Swagger. FastAPI được lựa chọn nhờ khả năng xử lý bất đồng bộ (asynchronous), phù hợp với các hệ thống cần phục vụ nhiều request đồng thời.

Hệ thống sử dụng **PostgreSQL** làm cơ sở dữ liệu quan hệ để lưu trữ các dữ liệu có cấu trúc như thông tin xe, người bán, giao dịch và metadata liên quan. PostgreSQL đảm bảo tính toàn vẹn dữ liệu, hỗ trợ các ràng buộc quan hệ và khả năng truy vấn phức tạp. Bên cạnh đó, **Qdrant** được sử dụng như một vector database, phục vụ cho bài toán tìm kiếm và gợi ý dựa trên vector embedding, cho phép hệ thống thực hiện các phép so sánh tương đồng (similarity search) hiệu quả trên không gian nhiều chiều.

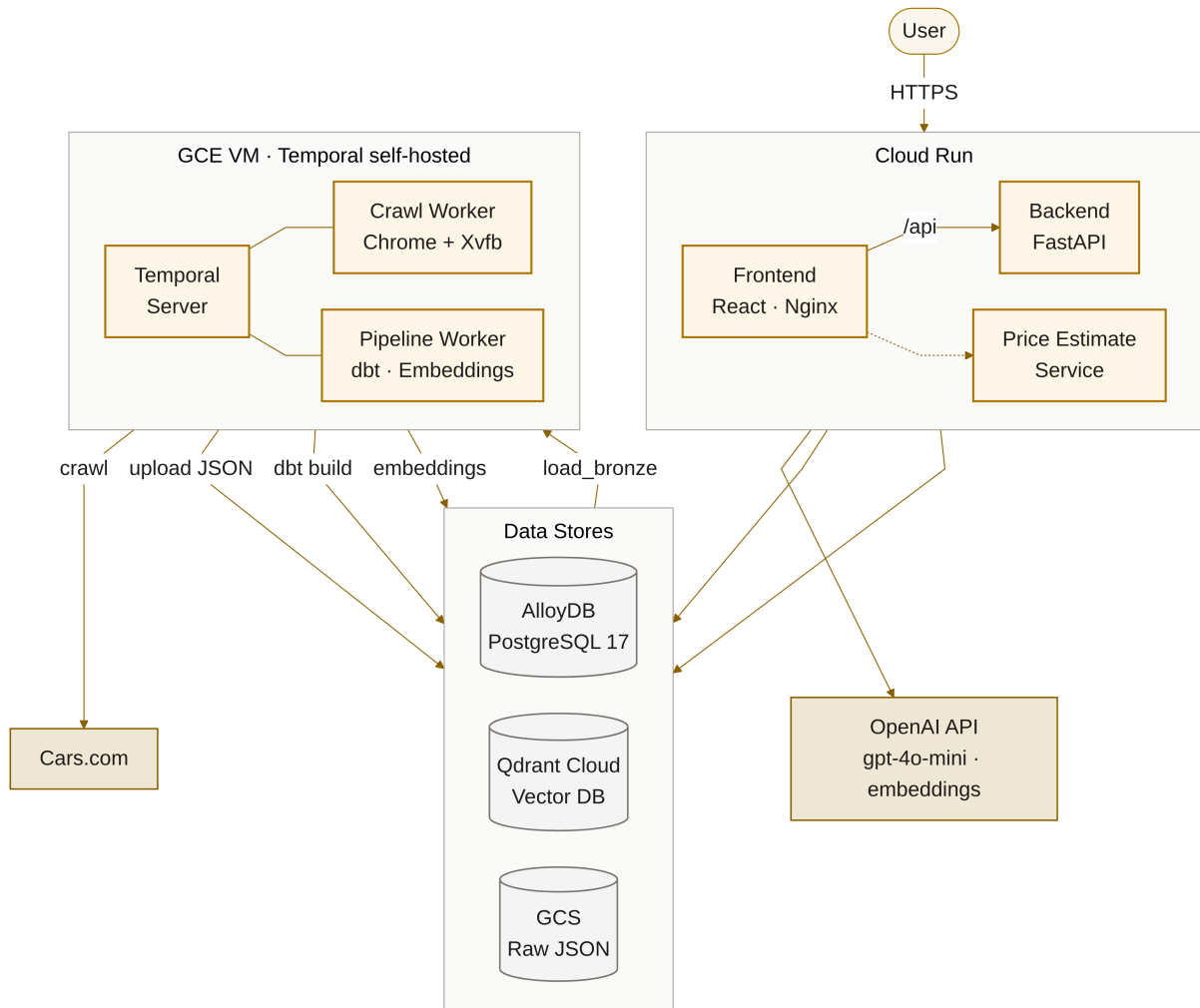
Hệ thống được thiết kế theo hướng **tách biệt dịch vụ**, trong đó mỗi thành phần đảm nhiệm một vai trò riêng và có thể triển khai, mở rộng độc lập. Trên môi trường sản xuất (Google Cloud Platform), Frontend và Backend là hai dịch vụ **Cloud Run** riêng biệt: Frontend (React, phục vụ qua Nginx) hiển thị giao diện tại tên miền *carsalesfinder.com* và ủy quyền các request /api sang Backend; Backend (FastAPI) xử lý nghiệp vụ, kết nối **AlloyDB** (PostgreSQL 17) qua kết nối SSL và **Qdrant Cloud** cho tìm kiếm vector. Cloud Run tự động co giãn số instance theo tải, giúp hệ thống không phải tự quản trị hạ tầng máy chủ.

Toàn bộ các service được đóng gói bằng **Docker**, nhờ đó môi trường phát triển cục bộ có thể tái lập đầy đủ hệ thống bằng Docker Compose (frontend cổng 3000, backend cổng 8000, PostgreSQL cổng 5432, Qdrant cổng 6333) với cùng một image như sản xuất. Cách tiếp cận này đảm bảo tính nhất quán giữa môi trường phát triển và triển khai, đồng thời tạo tiền đề thuận lợi cho việc mở rộng hoặc tích hợp thêm dịch vụ mới trong tương lai.

3.2.2 Kiến trúc tổng quan

Như minh họa trong Hình 3.1, hệ thống được thiết kế theo kiến trúc phân tầng (layered architecture) kết hợp với mô hình microservices nhằm đảm bảo tính tách biệt chức năng, khả năng mở rộng và dễ bảo trì. Toàn bộ hệ thống được chia thành ba lớp chính: *Client Layer*, *Backend Layer* và *Data Layer*.

Client Layer bao gồm ứng dụng frontend được xây dựng bằng React, được đóng gói tĩnh và phục vụ qua Nginx trên một dịch vụ Cloud Run riêng. Lớp này chịu trách nhiệm hiển thị giao diện người dùng, thu thập dữ liệu đầu



Hình 3.1: Kiến trúc tổng quan hệ thống

vào và gửi các yêu cầu đến backend thông qua các REST API. Client không xử lý logic nghiệp vụ phức tạp mà tập trung vào trải nghiệm người dùng và tương tác.

Backend Layer đóng vai trò trung tâm điều phối và xử lý nghiệp vụ của hệ thống. Backend được xây dựng trên nền tảng FastAPI, triển khai như một dịch vụ Cloud Run độc lập, cung cấp các endpoint RESTful cho frontend. Bên trong backend, các chức năng được chia thành nhiều service độc lập nhằm tuân thủ nguyên tắc single responsibility:

- *Authentication Service*: quản lý xác thực người dùng, đăng nhập, phân quyền và kiểm soát truy cập.
- *Search Service*: xử lý các yêu cầu tìm kiếm xe dựa trên tiêu chí lọc, từ khóa và các điều kiện truy vấn.
- *Recommendation Engine*: thực hiện logic gợi ý xe dựa trên hành vi người dùng và độ tương đồng dữ liệu, kết hợp truy vấn vector embedding.
- *Chatbot Service*: cung cấp khả năng tư vấn tự động, hỗ trợ người dùng thông qua hội thoại và tích hợp với mô hình ngôn ngữ lớn.

Các service trong backend có thể giao tiếp với nhau thông qua các lời gọi nội bộ và cùng truy cập các nguồn dữ liệu phía dưới, giúp hệ thống dễ dàng mở rộng hoặc bổ sung thêm service mới trong tương lai.

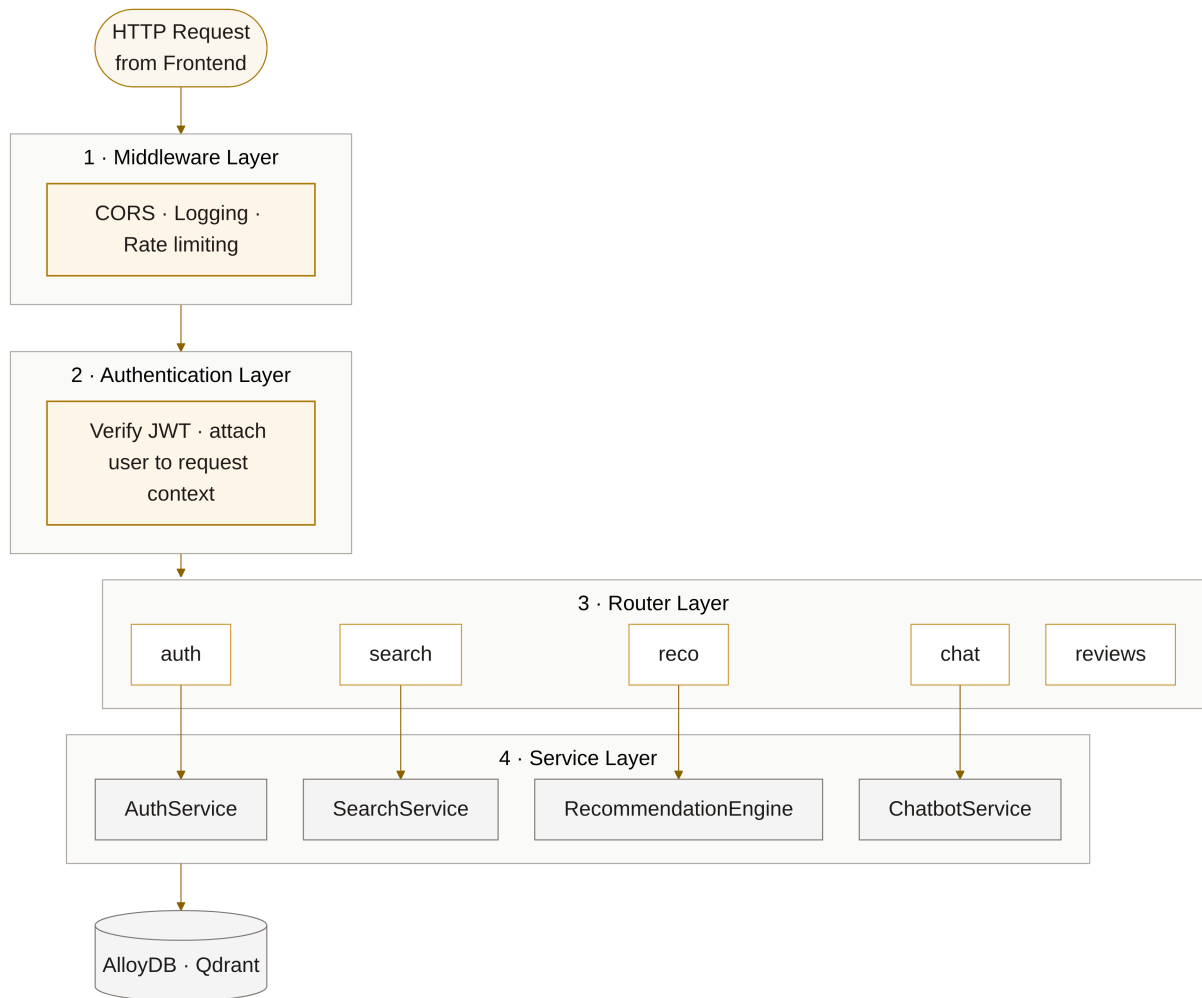
Data Layer chịu trách nhiệm lưu trữ và truy xuất dữ liệu, bao gồm nhiều thành phần chuyên biệt cho từng loại dữ liệu. PostgreSQL (AlloyDB trên môi trường sản xuất) được sử dụng làm cơ sở dữ liệu quan hệ để lưu trữ dữ liệu có cấu trúc như thông tin xe, người dùng và người bán. Qdrant đóng vai trò là vector database, phục vụ cho các bài toán tìm kiếm và gợi ý dựa trên độ tương đồng của vector embedding.

Ngoài ra, hệ thống còn tích hợp với các dịch vụ bên ngoài như OpenAI API để hỗ trợ chatbot và tạo embedding phục vụ cho Recommendation Engine. Việc tách biệt các dịch vụ bên ngoài giúp hệ thống linh hoạt trong việc thay thế hoặc mở rộng các nhà cung cấp khác trong tương lai.

Tất cả các thành phần trong hệ thống được đóng gói dưới dạng container Docker và triển khai trên hạ tầng serverless của Cloud Run, giao tiếp với nhau qua HTTPS. Cách tiếp cận này giúp đảm bảo tính nhất quán giữa các môi trường triển khai, đồng thời tạo nền tảng cho việc mở rộng hệ thống lên các hạ tầng orchestration phức tạp hơn như Kubernetes khi cần thiết.

3.2.3 Kiến trúc API

Backend API được tổ chức theo mô hình phân tầng với bốn lớp chính.



Hình 3.2: Kiến trúc phân tầng của Backend API

Middleware Layer là lớp đầu tiên tiếp nhận request, thực hiện kiểm tra CORS để cho phép cross-origin requests từ frontend, ghi log mọi request để theo dõi và debug, và áp dụng rate limiting để ngăn chặn lạm dụng API.

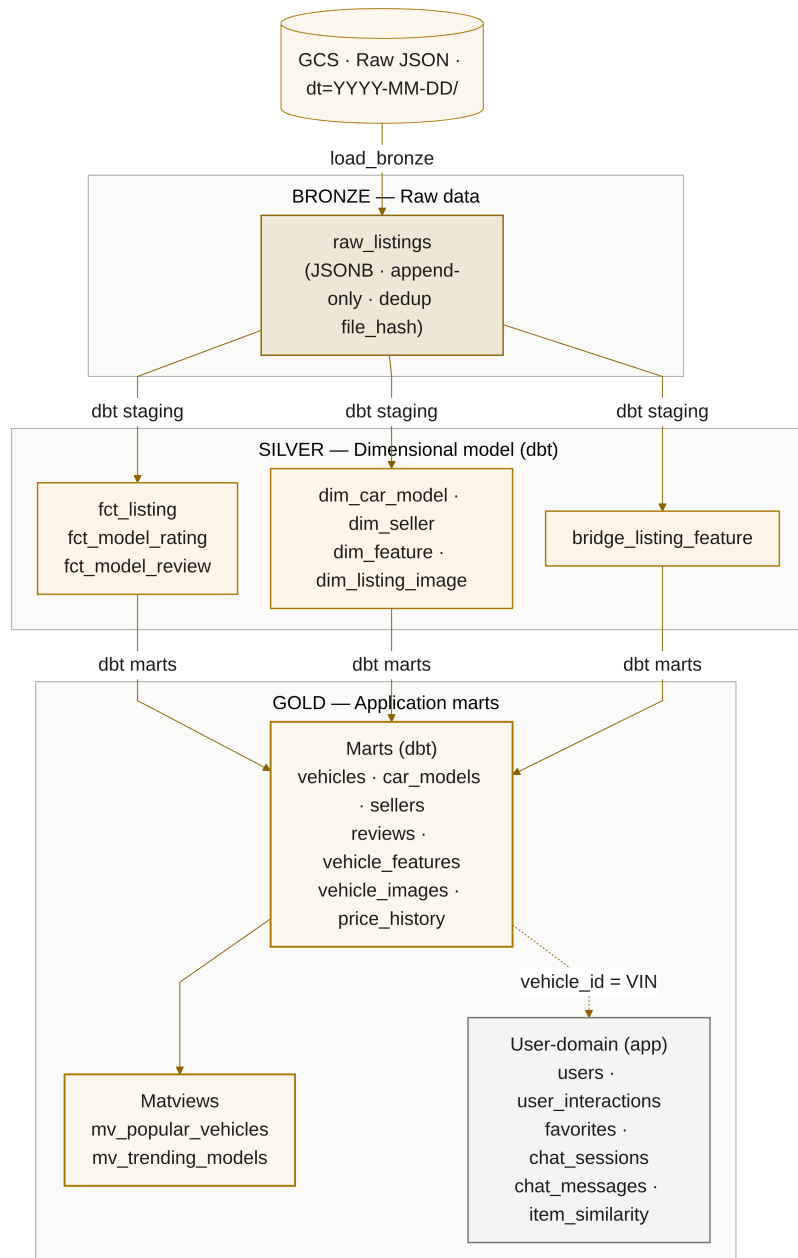
Authentication Layer xác thực JWT token trong header Authorization, giải mã và kiểm tra tính hợp lệ của token, sau đó gắn thông tin user vào context để các lớp sau sử dụng.

Router Layer định tuyến request đến endpoint tương ứng bao gồm auth cho đăng nhập và đăng ký, search cho tìm kiếm xe, reco cho gợi ý, chat cho chatbot, và reviews cho đánh giá. Mỗi router thực hiện validation dữ liệu đầu vào theo schema định nghĩa sẵn.

Service Layer chứa business logic của từng tính năng. AuthService xử lý hash password và tạo token. SearchService xây dựng query động và cache kết quả. RecommendationEngine tính toán similarity và rank gợi ý. ChatbotService thực hiện RAG workflow với vector database.

3.2.4 Thiết kế Database

Hệ thống sử dụng kiến trúc multi-schema theo mô hình **Medallion Architecture** với ba tầng dữ liệu: Bronze, Silver và Gold. Các bảng trong Silver và Gold được tạo và quản lý bởi **dbt** (data build tool), ngoại trừ các bảng user-domain do ứng dụng FastAPI ghi trực tiếp.



Hình 3.3: Kiến trúc multi-schema database

Bronze Schema lưu trữ dữ liệu thô trực tiếp từ quá trình crawling, chỉ gồm một bảng duy nhất `bronze.raw_listings`. Mỗi hàng chứa toàn bộ file JSON crawled dưới dạng cột payload (JSONB), kèm metadata như `file_hash` (SHA-256, khóa idempotent), `gcs_path`, `vin`, `crawl_date`. Bảng này là **append-only**: cùng một VIN crawl lại sẽ tạo hàng mới (khác `file_hash`), không ghi đè. Việc chọn bản ghi mới nhất được giải quyết ở tầng staging.

Silver Schema do dbt tạo, tổ chức theo mô hình chiều (Dimensional Model) với các bảng fact/dimension/bridge:

- **Bảng Fact:** `fct_listing` (bài đăng xe, incremental delete+insert), `fct_model_rating` (điểm đánh giá tổng hợp), `fct_model_review` (nhận xét chi tiết).
- **Bảng Dimension:** `dim_car_model` (hãng, dòng xe), `dim_seller` (thông tin đại lý), `dim_feature` (danh mục tính năng), `dim_listing_image` (hình ảnh xe).
- **Bảng Bridge:** `bridge_listing_feature` (quan hệ nhiều-nhiều giữa bài đăng và tính năng).

Gold Schema gồm hai nhóm bảng:

- **Bảng do dbt tạo (marts):** `vehicles` (denormalized, merge by VIN — bảng chính phục vụ frontend), `vehicle_price_history` (partitioned by month), `car_models`, `sellers`, `reviews`, `vehicle_features`, `vehicle_images`. Kèm theo các materialized view: `mv_popular_vehicles` và `mv_trending_models`.
- **Bảng do ứng dụng tạo (user-domain):** `users`, `user_interactions` (hành vi người dùng phục vụ recommendation), `user_favorites`, `user_searches`, `user_reviews`, `chat_sessions`, `chat_messages`, `item_similarity` (ma trận CF precomputed).

Lưu ý thiết kế: Trường `vehicle_id` trong các bảng user-domain là kiểu TEXT (VIN), **không có foreign key** đến `gold.vehicles`. Lý do: bảng `gold.vehicles` được dbt DROP/CREATE khi chạy full-refresh; một FK cross-schema sẽ phá vỡ quá trình rebuild. Tính toàn vẹn tham chiếu được đảm bảo thay thế bằng dbt test `assert_no_orphan_interactions`.

3.2.5 Data Pipeline

Quy trình chuyển đổi dữ liệu từ nguồn thô (crawled JSON trên GCS) đến trạng thái sẵn sàng phục vụ ứng dụng được điều phối hoàn toàn tự động bởi **Temporal** thông qua `WeeklyPipelineWorkflow`. Chi tiết từng giai đoạn đã được trình bày tại mục *Thu thập và xử lý dữ liệu* ở đầu chương này; phần này tóm tắt luồng xử lý từ góc nhìn backend.

Pipeline gồm ba workflow con chạy tuần tự (fail-stop):

1. **WeeklyCrawlWorkflow:** Thu thập dữ liệu từ Cars.com và upload JSON lên Google Cloud Storage theo cấu trúc phân vùng ngày `dt=YYYY-MM-DD/`.
2. **TransformWorkflow:** Nạp JSON vào `bronze.raw_listings` (idempotent qua `file_hash`), sau đó chạy dbt build để chuyển đổi qua các tầng staging → silver → gold theo Medallion Architecture.
3. **MLWorkflow:** Chạy song song ba tác vụ — tính ma trận `item_similarity` (cosine similarity), tạo vector embedding cho hệ thống đề xuất (`car_chatbot_vectors`), và tạo chunked embedding cho chatbot RAG (`car_vectorize`).

Toàn bộ pipeline chạy tự động hàng tuần, đảm bảo dữ liệu luôn được cập nhật mà không cần can thiệp thủ công. Tính chất **incremental** và **idempotent** cho phép chạy lại an toàn: bronze dedup bằng `file_hash`, gold merge by VIN, Qdrant upsert bằng `point_id = uuid5(vin)`.

3.2.6 Hệ thống Authentication

Hệ thống sử dụng JWT (JSON Web Token) cho xác thực người dùng.

Khi người dùng đăng ký, hệ thống nhận username, email và password từ frontend. Password được hash bằng bcrypt với salt ngẫu nhiên trước khi lưu vào database. Sau đó tạo JWT token chứa `user_id` với thời hạn 30 phút và trả về cho frontend.

Khi người dùng đăng nhập, hệ thống query user từ database theo username, verify password bằng cách so sánh hash, nếu đúng thì tạo và trả về JWT token mới.

Với mỗi request cần xác thực, frontend gửi token trong header Authorization. Backend extract token, decode và verify signature với SECRET_KEY, kiểm tra expiration time, sau đó query user từ database và gắn vào request context.

3.2.7 Tối ưu hiệu năng

Để đảm bảo hiệu năng, hệ thống kết hợp cache trong tiến trình (in-process) cho các tài nguyên tính và tối ưu truy vấn ở tầng cơ sở dữ liệu.

3.2.7.1 Cache trong tiến trình

Các tài nguyên có chi phí khởi tạo cao nhưng ít thay đổi được nạp một lần và lưu trong bộ nhớ tiến trình bằng `functools.lru_cache`, thay vì tính lại ở mỗi request:

- *Cấu hình hệ thống* gợi ý: file `reco_config.yaml` (trọng số, ngưỡng) được phân tích một lần và cache trong suốt vòng đời tiến trình.
- *Tài nguyên chatbot*: LLM client và vector store được khởi tạo một lần khi backend khởi động và tái sử dụng cho mọi lượt hội thoại.

Cách tiếp cận này phù hợp với cấu hình `max_instances = 1` của Cloud Run, giúp giảm độ trễ mà không cần một dịch vụ cache ngoài.

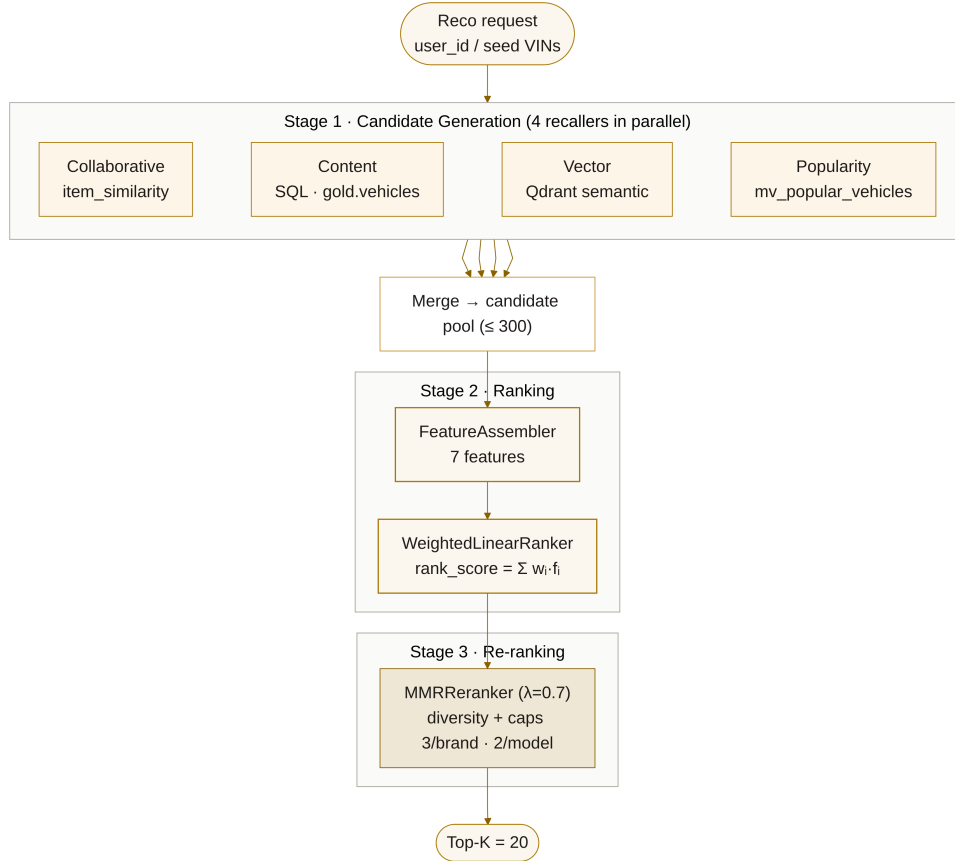
3.2.7.2 Tối ưu truy vấn Database

Các chỉ mục (index) được tạo trên các cột thường xuyên sử dụng trong điều kiện lọc và join. Cụ thể, bảng `gold.user_interactions` có index trên `user_id`, `vehicle_id`, `interaction_type` và `created_at`. Bảng `bronze.raw_listings` có GIN index trên cột `payload` (JSONB) để tăng tốc truy vấn JSON.

Đối với các truy vấn tổng hợp phức tạp, hệ thống sử dụng **materialized views** — `mv_popular_vehicles` (xe phổ biến theo lượt tương tác) và `mv_trending_models` (dòng xe được quan tâm nhiều nhất) — được refresh tự động mỗi lần pipeline chạy. Cách tiếp cận này giúp giảm chi phí tính toán tại thời điểm truy vấn và đảm bảo hiệu năng ổn định cho các API thường xuyên được gọi.

3.2.8 Hệ thống Recommendation

Recommendation Engine được thiết kế theo kiến trúc **3-stage hybrid pipeline**, xử lý tuần tự qua ba giai đoạn: candidate generation, ranking và re-ranking. Engine được khởi tạo mới cho mỗi request (stateless) — không fit model trong request, mà sử dụng các tín hiệu precomputed (item_similarity, vector embeddings) đã tính sẵn bởi pipeline ML.



Hình 3.4: Luồng hoạt động của Recommendation Engine

3.2.8.1 Stage 1 — Candidate Generation

Giai đoạn đầu tiên thu thập một tập ứng viên rộng từ **4 recaller chạy song song**, mỗi recaller khai thác một nguồn tín hiệu khác nhau:

- **CollaborativeRecaller:** Item-based Collaborative Filtering sử dụng bảng `gold.item_similarity` đã pre-computed. Với mỗi xe mà user đã tương tác (seed), hệ thống tra cứu K xe láng giềng tương tự nhất. Điểm số được tính có trọng số theo loại tương tác và time decay:

$$\text{score}_{\text{neighbor}} = \sum_{\text{seed}} w_{\text{type}} \cdot e^{-\lambda \cdot \text{days}} \cdot \text{similarity}$$

trong đó trọng số tương tác: view=1.0, click=2.0, compare=3.0, save/favorite=4.0, contact/inquiry=8.0; $\lambda = 0.05$ (half-life ≈ 14 ngày).

- **ContentRecaller:** Tìm xe tương tự dựa trên đặc tả kỹ thuật (brand, model, fuel_type, transmission, drive-train) và khung giá ($\pm 30\%$ giá xe seed). Sử dụng SQL query trực tiếp trên `gold.vehicles`, tính điểm tương đồng theo rule-based scoring:

$$\text{score} = 2.0 \cdot [\text{brand}] + 1.5 \cdot [\text{model}] + 0.5 \cdot [\text{fuel}] + 0.5 \cdot [\text{trans}] + 0.3 \cdot [\text{drive}] + 1.0 \cdot [\text{price_in_band}]$$

- **VectorRecaller:** Tìm kiếm ngữ nghĩa (semantic similarity) trên Qdrant collection `car_chatbot_vectors`. Truy xuất vector embedding đã lưu của xe seed, sau đó tìm các vector láng giềng gần nhất trong không gian 3072 chiều.
- **PopularityRecaller:** Xe phổ biến toàn hệ thống từ materialized view `mv_popular_vehicles`. Ưu tiên cùng brand với seed vehicle. Đây cũng là **fallback cho cold-start**: người dùng mới hoặc khách (guest) không có lịch sử tương tác sẽ nhận gợi ý hoàn toàn từ recaller này.

Các recaller hoạt động theo cơ chế **fail-soft**: nếu một nguồn dữ liệu không khả dụng (ví dụ: Qdrant tạm ngừng, matview chưa tồn tại), recaller trả về tập rỗng thay vì raise exception, đảm bảo hệ thống vẫn hoạt động với các nguồn còn lại.

3.2.8.2 Stage 2 — Ranking

Sau khi thu thập ứng viên, module `FeatureAssembler` hợp nhất kết quả từ 4 recaller, bổ sung thuộc tính xe từ `gold.vehicles` (brand, model, price, rating, crawled_at), và tính toán thêm 3 derived features:

- **price_fit:** Mức độ phù hợp với ngân sách user (= giá trung bình các xe đã xem). Giá càng gần budget, điểm càng cao (1.0 = khớp hoàn toàn, giảm tuyến tính khi lệch).
- **model_rating:** Điểm đánh giá dòng xe từ Cars.com (0–5, normalize về 0–1).
- **recency:** Độ mới của bài đăng, sử dụng exponential decay với half-life ≈ 30 ngày.

`WeightedLinearRanker` tính điểm tổng cho mỗi ứng viên bằng công thức:

$$\text{rank_score} = \sum_{i=1}^7 w_i \cdot f_i$$

với 7 features và trọng số cấu hình trong `reco_config.yaml`:

Bảng 3.2: Bầy đặc trưng và trọng số của `WeightedLinearRanker`

Feature	Weight	Nguồn
cf_score	3.0	CollaborativeRecaller
vector_score	2.0	VectorRecaller (Qdrant)
content_score	1.5	ContentRecaller (SQL)
popularity	1.0	PopularityRecaller (matview)
price_fit	1.0	Derived (budget proximity)
model_rating	0.8	Cars.com consumer rating
recency	0.5	Listing freshness

CF warmup: Trọng số `cf_score` được nhân thêm hệ số `cf_scale` $\in [0, 1]$ tỉ lệ thuận với số tương tác của user. Khi user mới (0 interactions), `cf_scale` = 0 và CF không đóng góp vào điểm. Khi user đạt ngưỡng 20 interactions, `cf_scale` = 1.0 và CF phát huy tối đa. Cơ chế này giúp hệ thống chuyển dịch tự nhiên từ content/popularity-driven sang collaborative-driven khi tích lũy đủ dữ liệu hành vi.

Ranker được thiết kế theo mô hình **weighted-linear** (thay vì black-box model) để đảm bảo tính giải thích được (explainability): có thể chỉ ra chính xác tín hiệu nào đẩy một xe lên cao trong danh sách.

3.2.8.3 Stage 3 — Re-ranking

`MMRReranker` (Maximal Marginal Relevance) cân bằng giữa **relevance** và **diversity** trong kết quả cuối cùng:

$$\text{MMR} = \lambda \cdot \text{relevance} - (1 - \lambda) \cdot \max_{\text{đã chọn}} \text{similarity}$$

với $\lambda = 0.7$ (nghiêng về relevance nhưng vẫn phạt trùng lặp). Similarity giữa hai xe được tính nhanh theo brand (0.5) và model (0.5) mà không cần embedding.

Ngoài ra, hệ thống áp dụng **hard caps** để tránh kết quả đơn điệu:

- Tối đa **3 xe** cùng brand trong danh sách cuối.
- Tối đa **2 xe** cùng car_model.

Khi tất cả ứng viên còn lại đều vi phạm caps, hệ thống *relax* ràng buộc và chọn theo relevance để đảm bảo đủ top_k kết quả (mặc định 20).

3.2.8.4 Tóm tắt luồng xử lý

Bảng 3.3: Tóm tắt ba giai đoạn của Recommendation Engine

Stage	Module	Mô tả
1	candidates.py	4 recaller song song $\rightarrow$ tập ứng viên ($\text{pool_size} \leq 300$)
2	features.py + ranker.py	Bổ sung 7 features, tính rank_score tuyến tính
3	reranker.py	MMR diversity + brand/model caps $\rightarrow$ top_k = 20

Toàn bộ trọng số và ngưỡng được cấu hình tập trung trong file `reco_config.yaml`, không có magic numbers trong code. Kiến trúc này cho phép điều chỉnh chiến lược gợi ý chỉ bằng cách thay đổi file YAML mà không cần sửa code.

3.3 Xây dựng chatbot tư vấn

3.3.1 Tổng quan và Đặt vấn đề

3.3.1.1 Đặt vấn đề

Trong bối cảnh thị trường ô tô phát triển sôi động, sự phong phú về lựa chọn giúp người mua có nhiều cơ hội tiếp cận các sản phẩm đa dạng, nhưng đồng thời cũng làm tăng sự khó khăn trong việc đánh giá và lựa chọn một chiếc xe thực sự phù hợp với nhu cầu cá nhân. Thông thường, khi tương tác với một website mua bán xe, người dùng phải tự thực hiện hàng loạt các thao tác lọc (filter) thủ công, tuần tự theo từng tiêu chí để có thể khoanh vùng được chiếc xe ưng ý. Quá trình lọc dựa trên biểu mẫu (form-based filtering) này đôi khi phức tạp, kém linh hoạt và khó đáp ứng được những câu hỏi mang tính tổng hợp, so sánh đa chiều của khách hàng.

Do đó, một chatbot có khả năng thấu hiểu ngôn ngữ tự nhiên, trực tiếp trả lời và tư vấn các lựa chọn phù hợp đã ra đời nhằm nhanh chóng đáp ứng nhu cầu đó. Thay vì phải chập vạt với các bộ lọc, người dùng có thể thoải mái trò chuyện và đưa ra các yêu cầu cụ thể để hệ thống tự động tìm kiếm.

Một Chatbot trong lĩnh vực tư vấn mua bán xe đòi hỏi về độ chính xác của các thông tin thực tế (thông số kỹ thuật, giá cả, tính trạng xe, ...). Việc chatbot cung cấp sai lệch thông tin có thể ảnh hưởng đến quyết định của người mua. Vì vậy, dựa trên dữ liệu các bài đăng hiện có trong website, nhóm đặt mục tiêu xây dựng một chatbot không chỉ giao tiếp tự nhiên, duy trì tốt ngữ cảnh hội thoại mà còn phải có cơ chế kiểm soát chặt chẽ, đảm bảo mọi câu trả lời luôn bám sát nguồn thông tin thực tế hiện có trong cơ sở dữ liệu.

Bên cạnh đó, để nâng cao trải nghiệm người dùng, chatbot được thiết kế tối ưu nhằm duy trì ngữ cảnh hội thoại. Hệ thống có khả năng ghi nhớ các câu hỏi trước đó và khai thác triệt để những thông tin đã được cung cấp, qua đó giúp hiểu rõ hơn về hành vi cũng như nhu cầu thực sự của người dùng trong suốt quá trình tương tác.

3.3.1.2 Phạm vi tư vấn của hệ thống

Hệ thống Chatbot tư vấn mua bán xe được thiết kế để giải quyết toàn diện các bước trong hành trình tìm kiếm và lựa chọn xe của khách hàng. Phạm vi hỗ trợ của Chatbot bao gồm các luồng chức năng chính sau:

- **Chào hỏi và thấu hiểu nhu cầu người dùng:** Chatbot có khả năng giao tiếp tự nhiên, duy trì ngữ cảnh hội thoại để chào đón khách hàng. Đặc biệt, thông qua cơ chế thu thập thông tin (Slot-filling), hệ thống chủ động đặt các câu hỏi dẫn dắt nhằm khai thác các tiêu chí cốt lõi như: ngân sách dự kiến, kiểu dáng xe, hãng xe ưa thích, hoặc mục đích sử dụng, từ đó phác họa rõ nét chân dung và nhu cầu thực sự của người dùng.

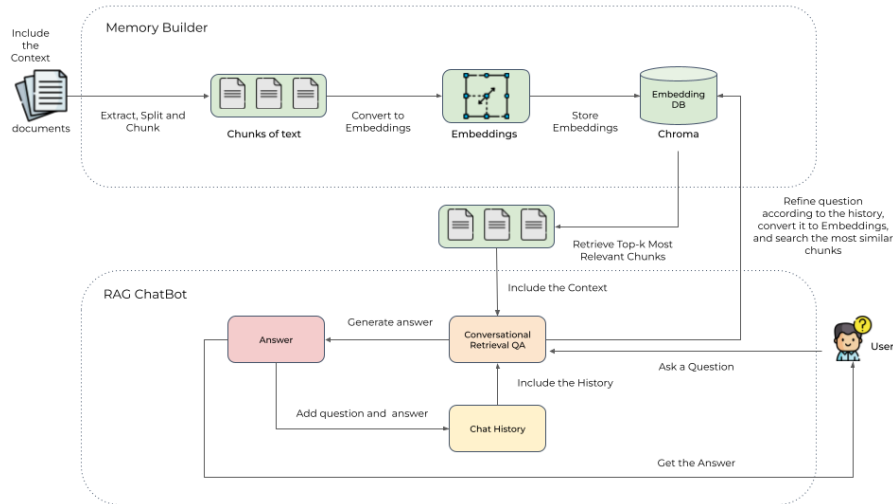
- **Gợi ý và so sánh xe theo nhu cầu:** Dựa trên các tập thông tin đã thu thập được, hệ thống sẽ tiến hành truy xuất dữ liệu và trả về danh sách các bài đăng phù hợp nhất. Chatbot không chỉ liệt kê mà còn tóm tắt các thông tin quan trọng của từng lựa chọn trong cùng tầm giá, phân tích các thông số và tính năng nổi bật. Điều này giúp người dùng dễ dàng so sánh, đánh giá ưu/nhược điểm để đưa ra quyết định phù hợp nhất.
- **Hỗ trợ trực quan và điều hướng mua hàng:** Nhằm nâng cao trải nghiệm thực tế, Chatbot có khả năng hiển thị trực tiếp hình ảnh của các mẫu xe được đề xuất thông qua đường dẫn ảnh (URL) lưu trữ trong hệ thống. Đồng thời, Chatbot cung cấp liên kết (link) dẫn trực tiếp đến trang chi tiết bài đăng gốc, giúp người dùng có góc nhìn trực quan nhất và dễ dàng thực hiện các thao tác chuyển đổi tiếp theo trên website.
- **Truy vấn thông số kỹ thuật chi tiết:** Khi người dùng có nhu cầu tìm hiểu sâu về một mẫu xe cụ thể, Chatbot đóng vai trò như một chuyên gia cung cấp chính xác các thông số kỹ thuật nổi bật (như động cơ, hiệu suất tiêu hao nhiên liệu, tính năng an toàn, nội/ngoại thất). Mọi thông tin đều được đảm bảo truy xuất trực tiếp từ cơ sở dữ liệu thực tế, loại bỏ rủi ro sai lệch thông tin.
- **So sánh trực quan đa chiều (Side-by-side Comparison):** Hệ thống hỗ trợ tính năng so sánh toàn diện giữa hai mẫu xe mục tiêu. Thông qua việc truy xuất đồng thời các dữ liệu thực tế (thông số kỹ thuật, mức giá, trang bị), Chatbot sẽ tự động tổng hợp, đối chiếu và phân tích chuyên sâu các ưu/nhược điểm của từng phương án. Điều này giúp cung cấp cho người dùng một góc nhìn khách quan, đa chiều để dễ dàng cân nhắc giữa các lựa chọn.

3.3.2 Cơ sở lý thuyết và Công nghệ áp dụng

3.3.2.1 Hạn chế của mô hình RAG truyền thống (Naive RAG)

Theo bài khảo sát chuyên sâu *Retrieval-Augmented Generation for Large Language Models: A Survey* [?], kiến trúc **Naive RAG** (RAG truyền thống) hoạt động theo một cơ chế tuyến tính cố định gọi là *Retrieve-Then-Read*. Trong cơ chế này, hệ thống chỉ thực hiện truy xuất dữ liệu một lần duy nhất dựa trên câu hỏi đầu vào, sau đó đưa ngay ngữ cảnh tìm được cho mô hình ngôn ngữ (LLM) để sinh câu trả lời. Tuy nhiên, cách tiếp cận có phần nhanh chóng này bộc lộ những điểm yếu chí mạng khi áp dụng vào các kịch bản tư vấn mua bán xe thực tế:

- **Thất bại trước các truy vấn phức tạp (Complex Queries):** Khi người dùng đặt những câu hỏi yêu cầu sự tổng hợp và đối chiếu thông tin từ nhiều nguồn, ví dụ: “*So sánh giá lẫn bánh và tiêu hao nhiên liệu của xe Mazda CX-5 và Hyundai Tucson*”, Naive RAG thường không thể đáp ứng. Nguyên nhân là do hệ thống thiếu khả năng chia nhỏ vấn đề (sub-querying) để tiến hành tìm kiếm từng dòng xe riêng biệt. Việc nhồi nhét quá nhiều thực thể vào một lượt tìm kiếm duy nhất dễ dẫn đến việc thiếu hụt dữ liệu hoặc cơ sở dữ liệu trả về kết quả sai lệch ngay từ lần thử đầu tiên.
- **Xử lý kém các truy vấn mơ hồ (Ambiguous Queries):** Trong thực tế, khách hàng thường bắt đầu bằng những câu hỏi rất chung chung và thiếu ngữ cảnh như: “*Tôi đang muốn mua xe, bạn hãy tư vấn giúp tôi được không?*”. Một nhân viên tư vấn con người sẽ lập tức chủ động hỏi ngược lại để khai thác nhu cầu (như ngân sách, kiểu dáng, loại nhiên liệu, ...). Ngược lại, vì bị bó buộc trong luồng tuyến tính, Naive RAG vẫn sẽ tiến hành tìm kiếm vector một cách máy móc dựa trên các từ khóa chung chung này. Hậu quả là hệ thống tự động trích xuất những đoạn tài liệu (context) ngẫu nhiên, không liên quan. Lượng thông tin dư thừa này tạo ra các thông tin nhiễu khiến LM mất phương hướng và trả lời một cách lan man và đặc biệt là lãng phí tài nguyên tính toán.



Hình 3.5: Luồng xử lý RAG truyền thống (Naive RAG)

3.3.2.2 Tổng quan về Agentic RAG

Định hướng giải pháp: Từ những khuyết điểm của mô hình Naive RAG đã phân tích, có thể thấy việc đơn thuần sử dụng một công cụ tìm kiếm cố định vào một Mô hình ngôn ngữ lớn (LLM) là không đủ để xây dựng một hệ thống tư vấn xe chuyên nghiệp và tối ưu. Để giải quyết triệt để bài toán này, đề án yêu cầu một kiến trúc chủ động và thông minh hơn: một hệ thống biết tự phân loại ý định người dùng, biết tự phân tách các bước tìm kiếm phức tạp, và đặc biệt là có khả năng chủ động tương tác (đặt câu hỏi ngược lại) nhằm làm rõ nhu cầu của khách hàng trước khi thực hiện truy xuất. Đây chính là động lực cốt lõi dẫn đến việc đề án lựa chọn và ứng dụng kiến trúc **Agentic RAG** (Mô hình RAG hướng tác tử).

Cơ chế hoạt động của Agentic RAG: Khác với luồng xử lý tuyến tính truyền thống, Agentic RAG ra đời như một sự kết hợp hoàn hảo giữa năng lực suy luận (Reasoning) của LLM Agent và độ chính xác của hệ thống truy xuất thông tin (Retrieval). Thay vì tin tưởng tuyệt đối vào một lần tìm kiếm duy nhất, Agentic RAG là một hệ thống tư duy thông qua các cơ chế cốt lõi sau:

- **Khả năng định tuyến và phân tách bài toán (Adaptive Routing & Query Decomposition):** Lấy cảm hứng từ kiến trúc *Adaptive-RAG* được đề xuất bởi Jeong và cộng sự (2024) [?], hệ thống loại bỏ luồng xử lý cố định mang tính máy móc cho mọi câu hỏi đầu vào. Thay vào đó, LLM Agent đóng vai trò như một bộ định tuyến thông minh để tự động phân tích mức độ phức tạp của truy vấn. Đối với các câu hỏi đa mục tiêu hoặc yêu cầu đối chiếu diện rộng, Agent sẽ tiến hành phân tách bài toán lớn thành chuỗi các câu hỏi phụ (sub-queries), thực hiện tìm kiếm nhiều bước (Multi-step retrieval) tuần tự hoặc song song để xử lý triệt để từng khía cạnh trước khi tổng hợp ngữ cảnh.
- **Cơ chế tự kiểm chứng và sửa lỗi (Self-Reflection & Critique Loops):** Áp dụng tư tưởng của *SELF-RAG* [?], hệ thống thiết lập các vòng lặp phản hồi (Feedback loops) nhằm loại bỏ rủi ro ảo giác (hallucination). Agent có khả năng tự đánh giá độ nhiễu của tài liệu trích xuất (để quyết định có cần viết lại câu hỏi và tìm kiếm lại không) cũng như tự phê bình câu trả lời của chính mình trước khi phản hồi cho khách hàng.
- **Điều hướng luồng công việc theo trạng thái (State-driven Workflows):** Kế thừa mô hình *StateFlow* [?], quá trình tư vấn được quản lý chặt chẽ qua các biến trạng thái (State). Hệ thống duy trì một bộ nhớ ngữ cảnh liên tục, cho phép Agent thực hiện các thao tác thu thập thông tin (Slot-filling) và luân chuyển mượt mà giữa các bước suy luận, hành động (Action) và quan sát kết quả (Observation).

Việc hiện thực hóa một kiến trúc Agentic RAG với các vòng lặp tuần hoàn (Cyclic Graph) và khả năng kiểm soát trạng thái phức tạp vượt quá giới hạn của các công cụ chuỗi (Chain) thông thường. Do đó, hệ sinh thái **LangGraph** đã được nhóm quyết định lựa chọn làm nền tảng kỹ thuật cốt lõi để vận hành mạng lưới tư vấn thông minh này.

3.3.2.3 LangGraph

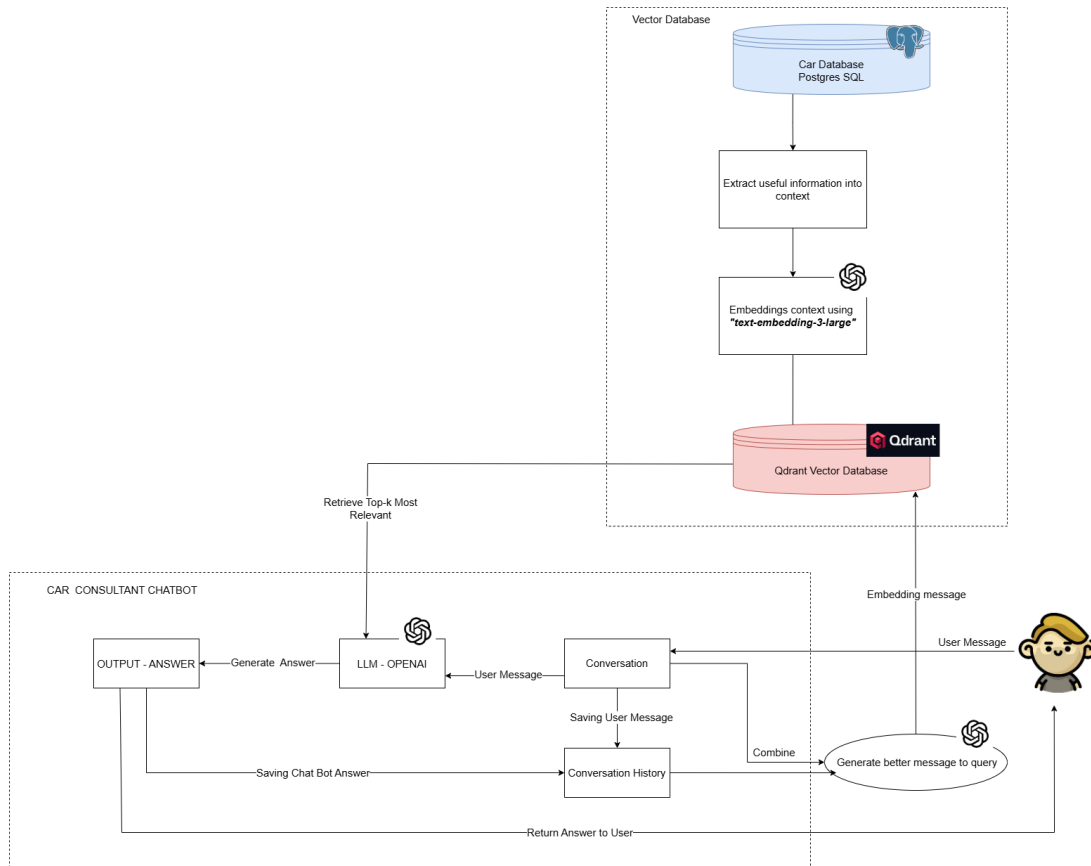
Để hiện thực hóa kiến trúc Agentic RAG với các cơ chế điều hướng phức tạp, chúng em quyết định lựa chọn khung **LangGraph** làm nền tảng công nghệ cốt lõi cho mô hình Agentic RAG.

- **Hiện thực hóa mô hình StateFlow một cách trực quan:** Lấy cảm hứng từ bài báo khoa học *StateFlow* [?], việc giải quyết các bài toán phức tạp bằng mô hình ngôn ngữ lớn (LLM) không nên để AI tự do suy luận vô hướng, mà cần được tổ chức thành một "máy trạng thái" (State Machine) chặt chẽ. LangGraph cung cấp kiến trúc hoàn hảo để xây dựng điều này bằng cách biểu diễn luồng công việc dưới dạng đồ thị. Cụ thể, các **Nodes (Nút)** sẽ đóng vai trò là các "trạm xử lý" (như trạm thu thập thông tin, trạm tìm kiếm xe), và các **Edges (Cạnh)** là các "quy tắc điều hướng". Nhờ vậy, lập trình viên có thể kiểm soát và định tuyến mạch lạc từng bước trong một phiên tư vấn mua bán xe, giống như đang xây dựng một quy trình chuẩn (SOP) cho một tư vấn viên thực thụ.
- **Phân tách rõ ràng giữa "Bộ nhớ" và "Hành động" (Trạng thái và Tác vụ):** Điểm sáng của LangGraph là cơ chế duy trì một không gian bộ nhớ chung (*State Machine Memory*) xuyên suốt toàn bộ cuộc hội thoại. Cơ chế này giúp chia tách rạch ròi hai quá trình: (1) Việc ghi nhớ ngữ cảnh (ngân sách, sở thích, hồ sơ của người dùng) và (2) Việc thực thi công cụ (như gọi lệnh SQL hay tìm kiếm Vector). Sự phân tách này giúp hệ thống Agentic RAG có thể thực hiện hàng loạt các tác vụ tìm kiếm phức tạp phía sau màn hình mà không làm xáo trộn hay đánh mất ngữ cảnh trò chuyện ban đầu. Từ đó, khắc phục triệt để tình trạng tràn bộ nhớ (prompt overflow) hay "quên" thông tin thường gặp ở các Agent truyền thống.
- **Triển khai nhanh chóng và khả năng gỡ lỗi (Debugging) mạnh mẽ:** Mặc dù mang sức mạnh kiểm soát luồng phức tạp, LangGraph lại sở hữu giao diện lập trình (API) rất thân thiện, sử dụng mã Python tiêu chuẩn và tích hợp sâu với hệ sinh thái mã nguồn mở LangChain. Đặc biệt, hệ thống này cung cấp sẵn cơ chế "lưu điểm xét duyệt" (*Checkpointing*). Tính năng này không chỉ cho phép ứng dụng tự động lưu trữ và khôi phục lại phiên trò chuyện của khách hàng bất cứ lúc nào, mà còn hỗ trợ lập trình viên theo dõi vết (tracing) từng bước suy luận của tác tử một cách minh bạch. Điều này giúp rút ngắn đáng kể thời gian từ khâu thiết kế lý thuyết đến khâu gỡ lỗi và triển khai sản phẩm thực tế.

3.3.3 Kiến trúc tổng thể

3.3.3.1 Sơ đồ kiến trúc tổng thể của hệ thống Agentic RAG

Kiến trúc tổng thể của hệ thống Agentic RAG được thiết kế để phân tách rõ ràng giữa quá trình xử lý dữ liệu nền tảng và quá trình tương tác thời gian thực với người dùng. Cụ thể, hệ thống bao gồm hai luồng hoạt động cốt lõi: luồng chuẩn bị dữ liệu (*Data Ingestion*) và luồng tư vấn trực tiếp (*Inference Workflow*).



Hình 3.6: Kiến trúc Agentic RAG trong hệ thống tư vấn mua bán xe

Quá trình vận hành của hệ thống được chia thành **hai luồng hoạt động độc lập** nhưng bổ trợ chặt chẽ cho nhau, cụ thể như sau:

1. **Luồng chuẩn bị dữ liệu (Data Ingestion):** Đây là tiến trình xây dựng cơ sở tri thức (Knowledge Base) cho Agent. Đầu tiên, các dữ liệu thô mang tính mô tả và thông số kỹ thuật của xe được trích xuất có chọn lọc từ cơ sở dữ liệu quan hệ (PostgreSQL). Sau đó, các đoạn văn bản quan trọng này được đưa qua mô hình nhúng (Embedding Model) để mã hóa thành các vector toán học trong không gian nhiều chiều. Cuối cùng, các vector này được lưu trữ và lập chỉ mục tại cơ sở dữ liệu vector chuyên dụng (Qdrant Database), tạo tiền đề vững chắc cho việc tìm kiếm độ tương đồng ngữ nghĩa sau này.
2. **Luồng tư vấn trực tiếp (Inference Workflow):** Đây là luồng tương tác trực tiếp theo thời gian thực (real-time) với khách hàng. Khi hệ thống tiếp nhận câu hỏi từ người dùng, Backend sẽ tiến hành khởi tạo phiên hội thoại, thiết lập công cụ LLM và kết nối tới các cơ sở dữ liệu. Lời gọi lệnh (prompt) và lịch sử hội thoại sau đó được đẩy vào trung tâm điều phối là đồ thị LangGraph. Tại đây, Tác tử (Agent) sẽ tự động phân tích ý định (*intent*) từ câu hỏi của người dùng để lập kế hoạch xử lý. Tùy thuộc vào chủ đích này, Agent có thể quyết định đưa ra phản hồi trực tiếp (đối với các giao tiếp thông thường) hoặc kích hoạt chiến lược truy xuất lai (Hybrid Retrieval). Chiến lược này là sự kết hợp chặt chẽ giữa truy vấn ngữ nghĩa mềm (Semantic Search) trên Qdrant để tìm kiếm các bài đăng phù hợp, và truy vấn điều kiện cứng (SQL Hard-filter) trên PostgreSQL để trích xuất chính xác các thông số định lượng (như giá cả, năm sản xuất). Cuối cùng, toàn bộ ngữ cảnh thu thập được sẽ được LLM tổng hợp, kiểm chứng và sinh ra câu trả lời tư vấn hoàn chỉnh nhất.

3.3.3.2 Quy trình thu thập và xử lý dữ liệu (Data Ingestion)

Quá trình chuẩn bị dữ liệu (Data Ingestion) cho chatbot đóng vai trò tiên quyết trong việc xây dựng cơ sở tri thức cho hệ thống, được thực hiện tuần tự qua bốn giai đoạn:

- **Thu thập và chuẩn hóa dữ liệu (Data Extraction & Normalization):** Dữ liệu gốc được truy xuất trực tiếp từ hệ quản trị cơ sở dữ liệu AlloyDB PostgreSQL. Cụ thể, hệ thống khai thác các bảng dữ liệu đã qua tinh chế thuộc tầng *Gold*, đảm bảo thông tin đầu vào đã được làm sạch, đồng nhất và tối ưu hóa cho các tác vụ phân tích quan trọng.
- **Tổng hợp nội dung mô tả (Content Generation):** Dựa trên tập kết quả truy vấn SQL, các trường thuộc tính cốt lõi (như hãng sản xuất, dòng xe, mức giá, số km đã đi, thông số động cơ, màu sắc và các tính năng nổi bật) được hệ thống tự động ghép nối thành các đoạn văn bản mô tả liền mạch, giàu ngữ nghĩa. Quá trình này đồng thời đính kèm các siêu dữ liệu (*metadata*) như mã VIN, phân khúc giá, loại hình nhiên liệu, ... nhằm phục vụ trực tiếp cho cơ chế lọc điều kiện cứng (Hard-filter) trong quá trình truy xuất lại sau này.
- **Phân mảnh văn bản (Text Chunking):** Để tối ưu hóa độ chính xác của hệ thống tìm kiếm vector, các đoạn mô tả xe được chia nhỏ thông qua thuật toán *RecursiveCharacterTextSplitter* với cấu hình tham số `chunk_size=250` và độ chồng chéo `chunk_overlap=30`. Chiến lược phân mảnh này giúp bảo toàn trọn vẹn ngữ cảnh liền kề giữa các khối thông tin, từ đó cải thiện đáng kể khả năng ánh xạ ngữ nghĩa khi người dùng đặt câu hỏi mô tả nhu cầu bằng ngôn ngữ tự nhiên.
- **Nhúng vector và Lưu trữ (Embedding & Storage):** Các phân mảnh văn bản (chunks) được đưa qua mô hình **text-embedding-3-large** của OpenAI để chuyển đổi thành các vector đặc trưng trong không gian 3072 chiều, sử dụng độ đo khoảng cách **Cosine** để tính toán độ tương đồng. Các vector này sau đó được cập nhật (*upsert*) vào collection `car_vectorize` trên cơ sở dữ liệu Qdrant. Để đảm bảo hệ thống luôn nắm bắt được tình trạng tồn kho mới nhất, tiến trình (pipeline) này được thiết lập chạy tự động định kỳ hàng tuần với cơ chế tái tạo (*recreate collection*), giúp ngăn chặn triệt để tình trạng tích lũy dữ liệu trùng lặp.

3.3.3.3 Quy trình tương tác với người dùng và Luồng xử lý hội thoại

Quy trình tương tác giữa người dùng và Chatbot tư vấn mua bán xe được thiết kế tối ưu nhằm đảm bảo tính toàn vẹn của dữ liệu ngữ cảnh, khả năng xử lý thời gian thực và trải nghiệm người dùng liền mạch. Chu trình xử lý toàn diện cho mỗi lượt hội thoại (Turn-by-turn lifecycle) được hiện thực hóa qua các giai đoạn mang tính hệ thống sau:

1. **Tiếp nhận và khởi tạo yêu cầu (Request Ingestion):** Luồng xử lý bắt đầu khi giao diện người dùng (tầng Frontend) gửi yêu cầu truy vấn đến máy chủ thông qua một giao thức API thiết lập sẵn. Tại tầng dịch vụ xử lý hội thoại của Backend, hệ thống thực hiện khởi tạo mô hình ngôn ngữ lớn (gpt-4o-mini) và thiết lập kết nối tới kho lưu trữ vector Qdrant. Tiến trình khởi tạo này chỉ thực hiện một lần duy nhất theo cơ chế bộ nhớ đệm toàn cục (Global Cache), giúp tối ưu hóa tài nguyên tính toán và giảm thiểu tối đa độ trễ phản hồi (Latency) cho các lượt truy vấn kế tiếp.
2. **Quản lý trạng thái và lưu trữ phiên hội thoại (Session & State Management):** Hệ thống áp dụng cơ chế quản lý phân cấp dựa trên định danh phiên (`session_id`). Đối với nhóm người dùng vãng lai (Guest), lịch sử hội thoại và hồ sơ nhu cầu tạm thời được duy trì trực tiếp trên bộ nhớ trong (*In-memory storage*). Ngược lại, đối với người dùng đã đăng nhập hệ thống, toàn bộ tiến trình hội thoại và các siêu dữ liệu liên quan được ghi nhận đồng bộ vào các bảng lưu trữ vật lý trong cơ sở dữ liệu.
3. **Chuẩn hóa truy vấn độc lập (Query De-contextualization):** Nhằm loại bỏ tính phụ thuộc ngữ cảnh của ngôn ngữ tự nhiên trong hội thoại nhiều lượt, hệ thống kích hoạt module xử lý ngôn ngữ đặc biệt. Thông qua một bộ lọc LLM chuyên biệt, hệ thống phân tích lịch sử trò chuyện gần nhất để tự động chuyển đổi câu hỏi phụ thuộc ngữ cảnh của khách hàng thành một truy vấn độc lập (*Standalone Question*). Giai đoạn này đảm bảo vector nhúng sinh ra phản ánh chính xác nhất chủ đích tìm kiếm thực tế khi thực hiện so khớp trên không gian vector tri thức.
4. **Vận hành đồ thị trạng thái LangGraph (State Graph Execution):** Câu hỏi sau khi chuẩn hóa cùng toàn bộ dữ liệu phiên được nạp làm trạng thái đầu vào (*Initial State*) để kích hoạt đồ thị trạng thái LangGraph. Quá trình xử lý nội tại của tác tử (Agent) lúc này được phân tách thành chuỗi các nút tác vụ (*Nodes*) tuần tự:

- *Cập nhật hồ sơ (update_profile)*: Tự động trích xuất các tiêu chí kỹ thuật và kinh tế của khách hàng (ngân sách dự kiến, kiểu dáng, nhiên liệu, hãng xe ưa thích hoặc danh sách hãng xe loại trừ) để cập nhật vào hồ sơ UserProfile của phiên hiện tại.
 - *Phân loại ý định (route_intent)*: Sử dụng cơ chế định hình đầu ra (*Structured Output*) của LLM để phân loại chính xác câu hỏi vào một trong bảy nhóm chủ đích cốt lõi (*compare, analytics, specs, specific, vague, chitchat, off_topic*).
 - *Định tuyến luồng công việc (Routing Edges)*: Dựa trên kết quả phân loại, hệ thống rẽ nhánh luồng xử lý sang các module chuyên biệt tương ứng (như so sánh đối chiếu xe, thống kê nền tảng, truy vấn thông số kỹ thuật chi tiết, hỏi thêm thông tin còn thiếu, hoặc chuyển hướng chủ đề hội thoại).
5. **Truy xuất lai và làm giàu ngữ cảnh (Hybrid Retrieval & Context Enrichment)**: Đối với các nhóm chủ đích cần khai thác kho tri thức nền tảng (như *specific* hoặc *vague*), nút tác vụ tìm kiếm lai (*hybrid_retrieve*) sẽ thực thi đồng thời hai tiến trình truy vấn song song:
- *Truy vấn lọc cứng (SQL Hard-filter)*: Thực hiện quét trực tiếp trên bảng dữ liệu xe tăng *Gold* (*gold.vehicles*) nhằm loại bỏ cơ học các thực thể không thỏa mãn điều kiện tiên quyết (khoảng giá, hãng sản xuất, loại nhiên liệu, tình trạng xe), giữ lại tối đa 6 bản ghi phù hợp.
 - *Truy vấn ngữ nghĩa mềm (Vector Semantic Search)*: Thực hiện tìm kiếm độ tương đồng Cosine trên bộ chỉ mục Qdrant (*car_vectorize*) dựa trên các mô tả định tính từ hồ sơ khách hàng (phong cách, tính năng mong muốn), áp dụng ngưỡng lọc điểm số tương đồng (*similarity score > 1.3*) để giữ lại tối đa 5 kết quả chất lượng tốt nhất.
- Tập ngữ cảnh thô sau đó tiếp tục được làm giàu cấu trúc bằng cách thiết lập liên kết khóa ngoại thông qua mã định danh VIN tới các bảng lưu trữ hình ảnh và tính năng mở rộng.
6. **Tổng hợp tri thức và sinh phản hồi tư vấn (Contextual Response Generation)**: Nút tác vụ sinh văn bản chuyên biệt nhận tập ngữ cảnh đã qua tinh chế và làm giàu để nạp vào mô hình *gpt-4o-mini*. Hệ thống áp dụng chiến lược giới hạn cửa sổ ngữ cảnh (chỉ duy trì tối đa 10 thông điệp gần nhất trong lịch sử) nhằm kiểm soát chi phí token và tránh hiện tượng nhiễu thông tin, từ đó sinh ra phản hồi tư vấn tự nhiên, khách quan và bám sát thực tế dữ liệu.
7. **Đồng bộ hóa kết quả và hiển thị trực quan (Response Rendering)**: Phản hồi hoàn chỉnh trả về từ API bao gồm nội dung văn bản tư vấn và mảng cấu trúc danh sách xe gợi ý (*vehicles[]*). Tại tầng giao diện (Frontend), nội dung tư vấn được tổ chức rõ ràng, đồng thời thông tin các mẫu xe được tách biệt hoàn toàn khỏi văn bản và hiển thị dưới dạng các thẻ sản phẩm trực quan (Vehicle Cards) chứa đầy đủ mã VIN, tiêu đề bài đăng, mức giá và hình ảnh thực tế. Quy trình thiết kế này đảm bảo chatbot không chèn trực tiếp các đường dẫn (URL) hỗn loạn vào nội dung nội văn, tối ưu hóa tính thẩm mỹ và trải nghiệm tương tác của người dùng.

3.3.4 Thiết kế luồng Agentic RAG bằng LangGraph

Mục này trình bày chi tiết thiết kế kiến trúc cốt lõi của chatbot. Điểm khác biệt mang tính quyết định so với mô hình RAG tuyến tính (Naive RAG) là hệ thống ứng dụng **LangGraph StateGraph** để điều phối luồng công việc một cách linh hoạt dựa trên ý định (*intent*) của người dùng. Trong đồ thị này, mỗi nhánh xử lý sẽ tương ứng với một đường ống truy xuất (*retrieval pipeline*) và chỉ thị hệ thống (*system prompt*) độc lập, chuyên biệt cho từng loại câu hỏi.

3.3.4.1 Khởi tạo tài nguyên và Chu trình thực thi (Execution Cycle)

Để tối ưu hóa tài nguyên tính toán và giảm thiểu độ trễ, các thành phần lõi của hệ thống được thiết lập theo cơ chế khởi tạo một lần (Singleton) và lưu trữ trong bộ nhớ đệm toàn cục (*Global Cache*). Các thành phần này bao gồm:

- **Mô hình ngôn ngữ (LLM):** Sử dụng gpt-4o-mini với tham số temperature=0.5 nhằm cân bằng giữa tính sáng tạo trong giao tiếp và tính chính xác về mặt số liệu.
- **Mô hình nhúng (Embedding Model):** Sử dụng text-embedding-3-large của OpenAI để biểu diễn văn bản thành vector.
- **Cơ sở dữ liệu Vector:** Kết nối đến Qdrant phục vụ truy vấn mềm và tìm kiếm ngữ nghĩa.
- **Đồ thị Tác tử (Agentic Graph):** Được biên dịch sẵn từ các Nodes và Edges thiết lập trước.

Trong quá trình vận hành, mỗi lượt tương tác của người dùng sẽ kích hoạt một chu trình xử lý chuẩn hóa bao gồm các bước:

1. **Tiền xử lý và Chuẩn hóa câu hỏi:** Trước khi đưa vào đồ thị, hệ thống sử dụng một bộ lọc LLM để phân tích lịch sử hội thoại gần nhất, qua đó chuyển đổi câu hỏi ẩn ý của người dùng thành một câu hỏi độc lập ngữ cảnh (*Standalone Question*).
2. **Khôi phục và nạp ngữ cảnh khách hàng (User Profile Ingestion):** Hệ thống tiến hành trích xuất hồ sơ UserProfile hiện tại từ bộ nhớ tạm (*in-memory storage*) dựa trên định danh phiên (*session_id*). Tiến trình này giúp tái dựng toàn bộ trạng thái nhu cầu, sở thích và các tiêu chí phân khúc định lượng đã thu thập từ các lượt tương tác trước đó, thiết lập không gian ngữ cảnh cá nhân hóa (*personalized context*) trước khi nạp vào đồ thị tác tử.
3. **Thực thi đồ thị (Graph Invocation):** Lời gọi truy vấn cùng trạng thái ban đầu được truyền vào LangGraph để chạy qua chuỗi các nút tác vụ (phân loại ý định, truy xuất, sinh văn bản).
4. **Cập nhật và Lưu trữ trạng thái:** Sau khi đồ thị kết thúc luồng, hồ sơ người dùng (với các dữ kiện mới thu thập) sẽ được lưu lại. Đồng thời, bộ nhớ hội thoại được cập nhật và giữ lại tối đa 10 tin nhắn gần nhất để tránh tràn giới hạn token.
5. **Phản hồi:** Trả về câu trả lời cuối cùng kèm theo danh sách các thẻ xe (*vehicles*) hiển thị trên giao diện.

3.3.4.2 Định nghĩa Trạng thái Hội thoại (AgenticState)

Trong kiến trúc LangGraph, dữ liệu không được truyền đi một cách rời rạc mà được quản lý tập trung thông qua một đối tượng trạng thái chung gọi là *AgenticState*. Được thiết kế dưới dạng cấu trúc dữ liệu định kiểu (*TypedDict*), trạng thái này đóng vai trò như "bộ nhớ làm việc" (*Working Memory*) của hệ thống, liên tục được cập nhật khi luồng dữ liệu đi qua các Nodes.

Bảng 3.4 mô tả chi tiết các không gian thông tin được lưu trữ và luân chuyển bên trong *AgenticState*:

Bảng 3.4: Cấu trúc biến trạng thái AgenticState trong LangGraph

Trường dữ liệu	Mô tả chức năng
messages	Lưu trữ chuỗi lịch sử hội thoại liên tục giữa người dùng (HumanMessage) và hệ thống (AIMessage).
question	Ghi nhận câu hỏi gốc dạng thô (raw input) từ người dùng.
query	Lưu trữ câu hỏi đã được chuẩn hóa độc lập (Standalone question), tối ưu riêng cho tác vụ tìm kiếm vector trên Qdrant.
context	Chứa toàn bộ ngữ cảnh được hệ thống truy xuất (Bao gồm dữ liệu SQL, kết quả Vector, đặc tả tính năng và đường dẫn hình ảnh).
answer	Lưu trữ văn bản phản hồi cuối cùng do Generator Node sinh ra để trả về cho người dùng.
intent	Nhân phân loại ý định cốt lõi của lượt hội thoại (được map vào 1 trong 7 danh mục định sẵn).
brand, model	Các thực thể (Entities) về hãng sản xuất và mẫu xe được bộ định tuyến trích xuất tự động từ câu hỏi.
session_id	Khóa định danh duy nhất (Unique ID) để theo dõi và quản lý phiên làm việc của từng khách hàng.
profile	Đối tượng UserProfile chứa các nhu cầu đã được thu thập (ngân sách, màu sắc, ...) dưới định dạng từ điển (dict).
vehicles	Mảng dữ liệu cấu trúc chứa danh sách các xe đề xuất, phục vụ việc hiển thị trực quan (Vehicle Cards) trên giao diện Frontend.

3.3.4.3 Cơ chế quản lý hồ sơ khách hàng và Thu thập thông tin (Slot-filling)

Trong các hệ thống đối thoại thông minh phức tạp, việc thấu hiểu sâu sắc nhu cầu người dùng đòi hỏi một cơ chế lưu trữ và cập nhật trạng thái liên tục xuyên suốt phiên làm việc. Để hiện thực hóa điều này, hệ thống tích hợp module quản lý hồ sơ khách hàng chuyên biệt, vận hành theo cơ chế thu thập thông tin định hướng mục tiêu (Slot-filling).

Hồ sơ người dùng (UserProfile) được khởi tạo và duy trì cô lập theo từng định danh phiên (session_id) trực tiếp trên bộ nhớ tạm thời của máy chủ - đảm bảo tính toàn vẹn dữ liệu khi hệ thống phải xử lý đồng thời nhiều yêu cầu từ lượng lớn khách hàng truy cập đồng thời.

3.3.4.3.1 Cấu trúc của Hồ sơ khách hàng

Không gian thông tin bên trong hồ sơ UserProfile không lưu trữ phẳng mà được cấu trúc phân cấp mạch lạc thành ba phân vùng chức năng nhằm phục vụ tối ưu cho cả hai chiến lược truy xuất điều kiện cứng (SQL Hard-filter) và tìm kiếm ngữ nghĩa mềm (Vector Search):

- **Các thuộc tính cốt lõi (Core Slots):** Nhóm các dữ kiện định lượng bắt buộc phải thu thập để thiết lập bộ lọc cơ học cho kho dữ liệu xe, bao gồm: ngưỡng ngân sách tối thiểu (budget_min), ngưỡng ngân sách tối đa (budget_max), kiểu dáng hộp số/thân xe (body_type), loại hình nhiên liệu sử dụng (fuel_type), thương hiệu sản xuất (brand) và tình trạng xe cũ/mới (condition).
- **Các tùy chọn mềm định tính (Soft Preferences):** Nhóm các tiêu chí mang tính trải nghiệm cá nhân của khách hàng, được sử dụng làm đầu vào cho mô hình nhúng nhằm so khớp không gian vector ngữ nghĩa mềm trên Qdrant, bao gồm: danh sách các tính năng công nghệ/an toàn mong muốn (features[]) và phong cách thiết kế hoặc cảm giác lái mục tiêu (vibe).
- **Dữ liệu bổ trợ ngữ cảnh mở rộng:** Phân vùng lưu trữ hành vi động phục vụ cho việc tinh chỉnh prompt và cá nhân hóa tư vấn, bao gồm: danh sách các thương hiệu khách hàng chủ động loại trừ khỏi phạm vi tìm kiếm (excluded_brands) và lịch sử các mẫu xe mà người dùng đã tiến hành xem chi tiết trong phiên hội thoại (viewed_models).

3.3.4.3.2 Quá Trình Cập Nhật

Nút tác vụ cập nhật hồ sơ (`update_profile`) được cấu trúc là điểm tiếp nhận đầu vào tiên quyết (Entry Node) trong mỗi request nạp vào đồ thị trạng thái LangGraph. Tác vụ xử lý tại nút này tuân theo một chu trình khép kín:

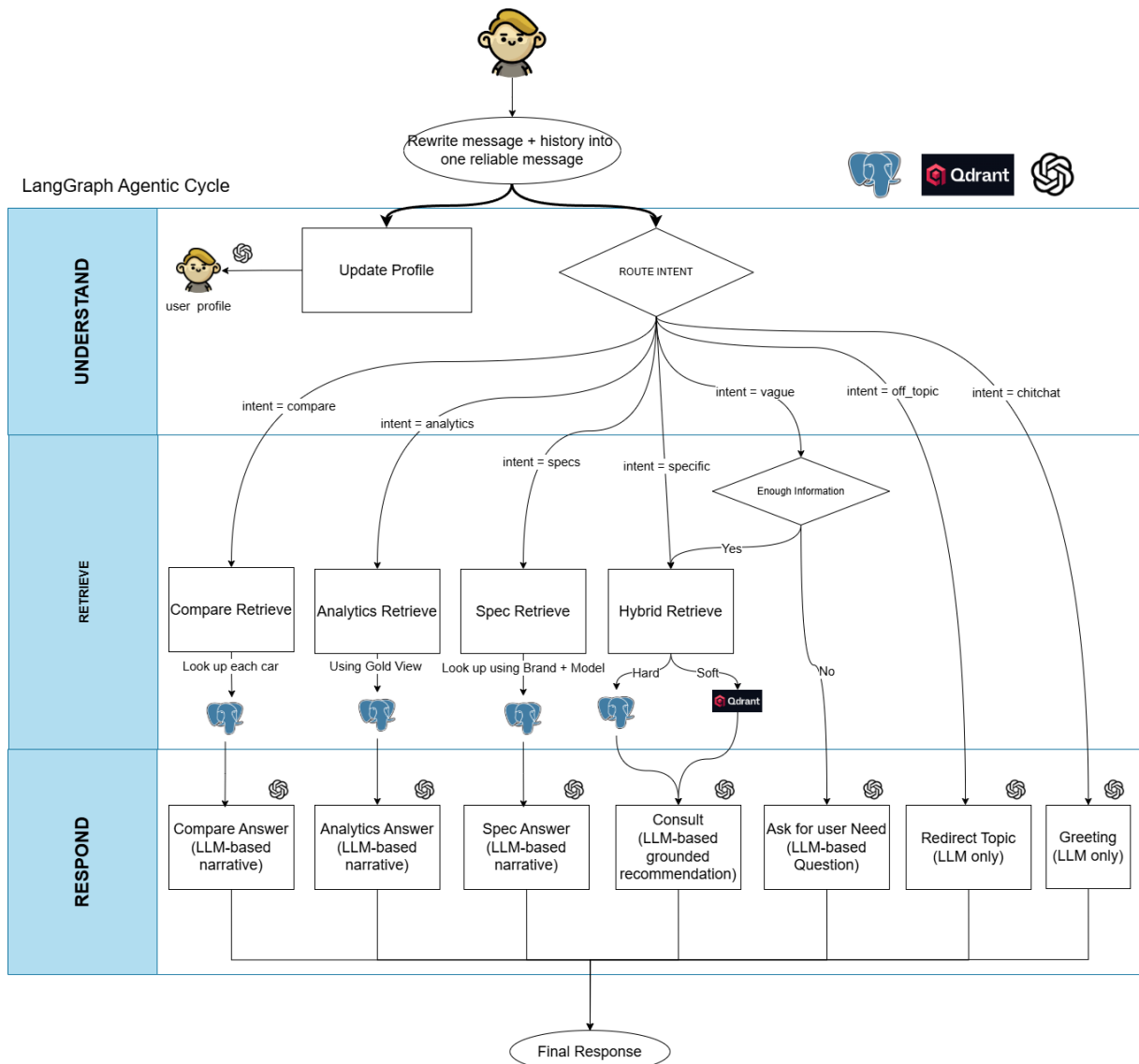
1. **Trích xuất cấu trúc văn bản (Information Extraction):** Ngay khi nhận được thông điệp mới từ người dùng, hệ thống sử dụng một mô hình ngôn ngữ lớn kết hợp cơ chế định hình cấu trúc đầu ra (*LLM Structured Output*) dựa trên lược đồ chỉ định sẵn (`ProfileUpdate`). Quá trình này giúp cô lập, bắt giữ và chuyển đổi các thực thể thông tin tự nhiên xuất hiện trong câu chat thành các dữ liệu khóa được chuẩn hóa.
2. **Hợp nhất dữ liệu tích lũy (Progressive Merging):** Các thực thể thông tin mới sau khi trích xuất sẽ được đối chiếu và hợp nhất vào hồ sơ hiện hành thông qua một cơ chế xử lý linh hoạt. Cụ thể, hệ thống sẽ tự động *ghi đè* (*overwrite*) các giá trị cũ nếu phát hiện người dùng thay đổi ý định đối với các tham số cốt lõi (ví dụ: điều chỉnh mức ngân sách từ 500 triệu lên 700 triệu). Ngược lại, đối với các tham số tùy chọn, hệ thống áp dụng cơ chế thêm vào (*append*) để liên tục mở rộng danh sách tiêu chí (ví dụ: bổ sung thêm yêu cầu "cửa sổ trời" vào mảng tính năng mong muốn). Phương pháp tiếp cận này giúp duy trì tính nhất quán của dữ liệu, đồng thời ngăn chặn triệt để hiện tượng suy giảm hoặc mất dấu ngữ cảnh lịch sử trong các phiên hội thoại đa lượt.

3.3.4.3.3 Hỗ trợ cơ chế kiểm soát điều kiện tối thiểu khi tư vấn gợi ý xe

Để ngăn chặn việc truy xuất dữ liệu vô hướng gây lãng phí tài nguyên khi người dùng có ý định tìm kiếm mơ hồ (*vague intent*), hệ thống thiết lập cơ chế kiểm soát dựa trên ba thuộc tính bắt buộc: `CORE_SLOTS = {budget_max, body_type, fuel_type}`. Sẽ có một cơ chế kiểm tra hồ sơ hiện tại; nếu khuyết thiếu bất kỳ thông số nào, luồng công việc sẽ lập tức rẽ nhánh sang trạng thái `ask_slot`. Tại đây, hệ thống tạm hoãn truy xuất kho xe và chủ động sinh câu hỏi dẫn dắt để thu thập trọn vẹn nhu cầu của khách hàng.

3.3.4.4 Kiến trúc Sơ đồ Đồ thị LangGraph

Hệ thống Agentic RAG được mô hình hóa dưới dạng một đồ thị có hướng không tuần hoàn (DAG) thông qua framework LangGraph, bao gồm tổng cộng **12 nút tác vụ (nodes)**. Trong kiến trúc này, nút `update_profile` được thiết lập làm điểm tiếp nhận đầu vào (Entry Point) cho mọi lượt hội thoại. Hình 3.7 minh họa chi tiết mạng lưới các nút xử lý và các luồng điều hướng rẽ nhánh bên trong hệ thống.



Hình 3.7: Sơ đồ luồng xử lý và định tuyến rẽ nhánh trong mạng đồ thị LangGraph

3.3.4.1 Cấu trúc các Nút xử lý (Nodes)

Mỗi nút trong đồ thị đại diện cho một tác vụ tính toán độc lập, được đóng gói (encapsulated) với các chỉ thị (*system prompt*) và đường ống xử lý riêng biệt. Bảng 3.5 trình bày chức năng cốt lõi của từng nút tác vụ trong hệ thống:

Bảng 3.5: Đặc tả chức năng các nút tác vụ (Nodes) trong đồ thị LangGraph

Nút tác vụ (Node)	Vai trò và Chức năng
update_profile	Cập nhật và hợp nhất dữ liệu vào hồ sơ UserProfile. Đây là nút khởi đầu cho mọi chu trình xử lý.
route_intent	Sử dụng LLM để phân tích ngữ nghĩa, phân loại ý định (<i>intent</i>) của câu hỏi và trích xuất các thực thể định danh (hãng sản xuất, mẫu xe).
ask_slot	Chủ động đặt câu hỏi để thu thập thêm dữ kiện khi hồ sơ khách hàng bị khuyết thiếu các tiêu chí bắt buộc (Core slots).
hybrid_retrieve	Thực thi truy xuất lai song song: Lọc điều kiện cứng cơ học (SQL Hard-filter) kết hợp tìm kiếm ngữ nghĩa mềm (Qdrant Semantic Search).
consult	Nút sinh văn bản cốt lõi, tổng hợp ngữ cảnh và đưa ra phản hồi tư vấn có cơ sở (<i>Grounded Generation</i>), loại trừ ảo giác.
compare_retrieve compare_answer	→ Nhánh chuyên biệt phục vụ tác vụ truy xuất song song nhiều thực thể và phân tích so sánh đối chiếu giữa 2 hoặc nhiều mẫu xe.
analytics_retrieve analytics_answer	→ Nhánh chuyên biệt xử lý các câu hỏi mang tính thống kê, phân tích dữ liệu tổng quan trên toàn bộ nền tảng.
spec_retrieve spec_answer	→ Nhánh chuyên biệt dùng để truy xuất và trả lời các thông số kỹ thuật chi tiết của một mẫu xe cụ thể.
redirect_topic	Xử lý các câu hỏi lạc đề (off-topic), đưa ra phản hồi từ chối lịch sự và chuyển hướng người dùng quay lại miền tri thức ô tô.

3.3.4.4.2 Cơ chế Định tuyến rẽ nhánh (Routing Logic)

Điểm làm nên sự linh hoạt của kiến trúc này là cơ chế điều hướng có điều kiện (*Conditional Edges*) được thực thi ngay sau khi nút `route_intent` hoàn tất phân tích ý định (thông qua hàm `route_after_intent`). Hệ thống sẽ tự động rẽ nhánh luồng công việc theo các kịch bản sau:

- **Nhóm truy vấn chuyên sâu:** Đối với các ý định đòi hỏi kỹ thuật truy xuất đặc thù như `compare` (so sánh), `analytics` (thống kê) hoặc `specs` (thông số), luồng công việc sẽ được định tuyến thẳng tới các đường ống xử lý tương ứng (`*_retrieve` → `*_answer` → END). Điều này giúp tối ưu hóa *system prompt* riêng cho từng tác vụ hẹp.
- **Nhóm tư vấn tìm kiếm xe:**
 - Khi ý định là *specific* (tìm kiếm cụ thể, rõ ràng): Hệ thống lập tức kích hoạt luồng `hybrid_retrieve` → `consult` → END để tổng hợp kết quả.
 - Khi ý định là *vague* (mơ hồ, cần gợi ý chung chung): Hệ thống sẽ kiểm tra tình trạng của hồ sơ. Nếu UserProfile đã tích lũy đủ các tiêu chí bắt buộc (Core slots), luồng `hybrid_retrieve` → `consult` mới được phép thực thi. Ngược lại, nếu thiếu dữ kiện, hệ thống sẽ rẽ nhánh sang nút `ask_slot` → END để tạm hoãn truy xuất và yêu cầu cung cấp thêm thông tin.
- **Nhóm giao tiếp ngoại lệ:**
 - Khi ý định là *chitchat* (giao tiếp xã giao thông thường): Hệ thống bỏ qua hoàn toàn bước truy xuất tri thức và đi thẳng đến nút `consult` để đưa ra phản hồi giao tiếp trực tiếp, giúp tiết kiệm chi phí token.
 - Khi ý định là *off_topic* (ngoài phạm vi tư vấn): Luồng dữ liệu được định tuyến sang `redirect_topic` → END nhằm định hướng lại hành vi của người dùng.

3.3.4.5 So sánh với thiết kế SELF-RAG / Adaptive-RAG

Thiết kế thực tế **không** triển khai các node *Grader*, *Hallucination/Critique*, *Query Rewriting* hay vòng lặp `iteration_count`. Thay vào đó:

- **Định tuyến theo intent** (ý tưởng Adaptive-RAG): mỗi loại câu hỏi có pipeline retrieval và prompt riêng, tránh retrieval không cần thiết.
- **Slot-filling chủ động**: thay vì retrieval mơ hồ khi thiếu thông tin, hệ thống hỏi ngược (`ask_slot`).
- **Grounded generation**: mọi node `*_answer` và `consult` chỉ được phép dùng context đã truy xuất; SQL hard-filter đảm bảo số liệu định lượng chính xác.
- **Hybrid retrieval**: kết hợp SQL (chính xác) và vector (ngữ nghĩa) thay vì RRF — hai nguồn được ghép thành context text, không fusion ranking riêng.

Đây là kiến trúc **intent-driven multi-branch RAG** trên LangGraph: đồ thị có hướng, mỗi nhánh chuyên biệt, phù hợp bài toán tư vấn xe với nhiều loại câu hỏi khác nhau.

3.3.5 Đánh giá mô hình (Evaluation)

3.3.5.1 Mục tiêu và Phương pháp luận

Trong quá trình phát triển các hệ thống xử lý ngôn ngữ tự nhiên phức tạp, việc sử dụng con người để đánh giá thủ công từng câu trả lời là một phương pháp tốn kém, mang tính chủ quan và không khả thi khi mở rộng quy mô. Để giải quyết bài toán này, đồ án kế thừa tư tưởng từ nghiên cứu *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena* [?], chính thức áp dụng phương pháp **LLM-as-a-Judge** (sử dụng Mô hình Ngôn ngữ lớn làm Giám khảo) thông qua framework đánh giá chuyên sâu **DeepEval GEval**.

Cụ thể, hệ thống cấu hình mô hình `gpt-4o-mini` đóng vai trò làm giám khảo độc lập. Mô hình này thực hiện chấm điểm tự động các phản hồi của chatbot dựa trên hệ thống tiêu chí đánh giá (*custom rubrics*) được thiết kế riêng cho từng luồng nghiệp vụ. Điểm số được chuẩn hóa trên thang đo định lượng từ 0 đến 1. Quá trình đánh giá được thiết kế nhằm đáp ứng ba mục tiêu cốt lõi:

- **Kiểm chứng năng lực định tuyến (Routing Verification)**: Đánh giá độ nhạy bén và tính chính xác của tác tử (Agent) trong việc phân tích ý định, đảm bảo hệ thống rẽ đúng nhánh luồng xử lý (tư vấn chuyên sâu, giao tiếp xã giao, xử lý lạc đề, hoặc yêu cầu bổ sung thông tin).
- **Đánh giá chất lượng sinh văn bản (Response Quality)**: Đo lường hiệu năng của chatbot thông qua điểm số GEval. Đảm bảo câu trả lời đạt các tiêu chí khắt khe về mặt nghiệp vụ: chính xác, trung thực và bám sát tập ngữ cảnh thực tế đã được truy xuất (grounded context).

3.3.5.2 Quy trình Đánh giá

Để đảm bảo tính khách quan, tập dữ liệu kiểm thử được xây dựng bao gồm khoảng 80 kịch bản hội thoại (test cases), phân bổ rải rác và bao phủ toàn bộ các nhóm ý định (intent) đầu vào. Toàn bộ quy trình đánh giá được đóng gói gọn gàng bên trong môi trường RAG `validation/DeepEval.ipynb` và vận hành theo chu trình khép kín sau:

1. **Kích hoạt hệ thống (Invocation)**: Các test case được nạp tự động vào hệ thống thông qua việc gọi trực tiếp đến API thực tế - mô phỏng chính xác hành vi của người dùng cuối.
2. **Thực thi đồ thị (Graph Execution)**: Câu hỏi đi qua mạng đồ thị LangGraph, thực thi các chuỗi tác vụ truy xuất và sinh văn bản tư vấn.
3. **Lưu trữ bộ nhớ đệm (Response Caching)**: Nhằm tối ưu hóa chi phí API và cô lập pha thu thập dữ liệu với pha chấm điểm, toàn bộ phản hồi đầu ra cùng tập ngữ cảnh tương ứng.
4. **Chấm điểm và Trực quan hóa (Scoring & Rendering)**: Framework DeepEval GEval tiến hành đọc dữ liệu từ tệp bộ nhớ đệm, áp dụng các *prompt* giám khảo chuyên biệt để tính toán điểm số. Cuối cùng, hệ thống tự động xuất báo cáo tổng hợp và biểu đồ phân tích, cung cấp cái nhìn trực quan về chất lượng của luồng Agentic RAG.

3.3.5.3 Xây dựng Bộ dữ liệu kiểm thử (Test Dataset)

Để phục vụ quá trình đánh giá tự động, đồ án đã tiến hành xây dựng một bộ dữ liệu kiểm thử tiêu chuẩn bao gồm 80 kịch bản hội thoại. Mỗi kịch bản được cấu trúc chặt chẽ thông qua ba trường dữ liệu: truy vấn đầu vào của người dùng (`user_input`), luồng xử lý kỳ vọng (`expected_flow`), và dữ liệu tham chiếu chuẩn (`reference - ground truth`).

Cần lưu ý rằng framework GEval không so khớp từ vựng máy móc (word-by-word) mà đánh giá dựa trên ngữ nghĩa. Cụ thể, LLM Giám khảo sẽ sử dụng dữ liệu tại cột `reference` làm hệ quy chiếu. Nó sẽ đọc hiểu câu trả lời của chatbot, đối chiếu ý nghĩa với hệ quy chiếu này, rồi chấm điểm theo các tiêu chí (rubrics) đặc thù của từng luồng nghiệp vụ.

Bảng 3.6 trình bày chi tiết phân bố dữ liệu và cơ sở ánh xạ giữa luồng kiểm thử với các nút ý định kỹ thuật trong hệ thống LangGraph.

Bảng 3.6: Phân bố tập dữ liệu kiểm thử và cơ sở ánh xạ luồng nghiệp vụ

Luồng đánh giá nghiệp vụ (<code>expected_flow</code>)	Số lượng (n)	Luồng xử lý tương ứng (LangGraph Intents)
Tư vấn chuyên sâu (<code>consult</code>)	27	<code>analytics</code> , <code>specs</code> , <code>specific</code> , <code>vague</code> (đã đủ điều kiện slots), <code>compare</code>
Xử lý ngoại lệ (<code>off-topic</code>)	15	<code>off_topic</code>
Giao tiếp xã giao (<code>greeting</code>)	15	<code>chitchat</code>
Bổ sung thông tin (<code>need-more-info</code>)	23	<code>vague</code> (khuyết thiếu điều kiện core slots)
Tổng cộng	80	

3.3.5.4 Hệ thống Tiêu chí Đánh giá GEval theo Luồng Nghiệp vụ

Để đảm bảo tính chính xác, mỗi luồng nghiệp vụ kiểm thử được thiết lập một bộ chỉ thị chấm điểm (*custom rubrics*) riêng biệt trên framework DeepEval GEval thay vì áp dụng một tiêu chí chung chung.

Bảng 3.7 đặc tả các tham số đầu vào (`SingleTurnParams`) và hệ thống tiêu chí chấm điểm tương ứng cho từng kịch bản nghiệp vụ:

Bảng 3.7: Đặc tả các bộ tiêu chí chấm điểm và tham số đầu vào của GEval

Luồng nghiệp vụ	Tham số đầu vào	Tiêu chí chấm điểm chính (Rubric)
Tư vấn chuyên sâu (<code>consult</code>)	<code>INPUT</code> <code>ACTUAL_OUTPUT</code> <code>RETRIEVAL_CONTEXT</code>	Đánh giá độ chính xác thực tế (<i>factual correctness</i>) và hình thức trình bày. Kiểm tra tính cơ sở (<i>grounded generation</i>) dựa trên ngữ cảnh đã truy xuất để bắt lỗi ảo giác.
Xử lý ngoại lệ (<code>off-topic</code>)	<code>INPUT</code> <code>ACTUAL_OUTPUT</code>	Đánh giá năng lực nhận biết câu hỏi ngoài phạm vi, từ chối trả lời chuyên sâu và chủ động điều hướng người dùng quay lại miền tri thức ô tô.
Giao tiếp xã giao (<code>greeting</code>)	<code>INPUT</code> <code>ACTUAL_OUTPUT</code>	Đánh giá sự lịch sự, khả năng tự giới thiệu đúng vai trò trợ lý tư vấn xe và việc đưa ra câu hỏi mở để chủ động khai thác nhu cầu của khách hàng.
Bổ sung thông tin (<code>need-more-info</code>)	<code>INPUT</code> <code>ACTUAL_OUTPUT</code>	Kiểm tra năng lực dẫn dắt để thu thập các thuộc tính cốt lõi còn thiếu. Áp dụng quy tắc phạt nặng (trừ điểm) nếu chatbot cố tình gợi ý xe khi chưa đủ thông tin nền tảng.

3.3.5.5 Kết quả đánh giá

Sau quá trình thực thi và chấm điểm tự động toàn bộ 80 kịch bản kiểm thử thông qua framework DeepEval GEval, hệ thống đã trích xuất được các số liệu thống kê mô tả chi tiết cho từng luồng nghiệp vụ. Bảng 3.8 tổng hợp các

chỉ số thống kê cốt lõi, đồng thời Hình 3.8 minh họa trực quan phân phối điểm số của các luồng với ngưỡng cơ sở (baseline threshold) được thiết lập ở mức **0.7**.

Bảng 3.8: Kết quả đánh giá DeepEval GEval theo từng luồng nghiệp vụ (Thang điểm 0–1)

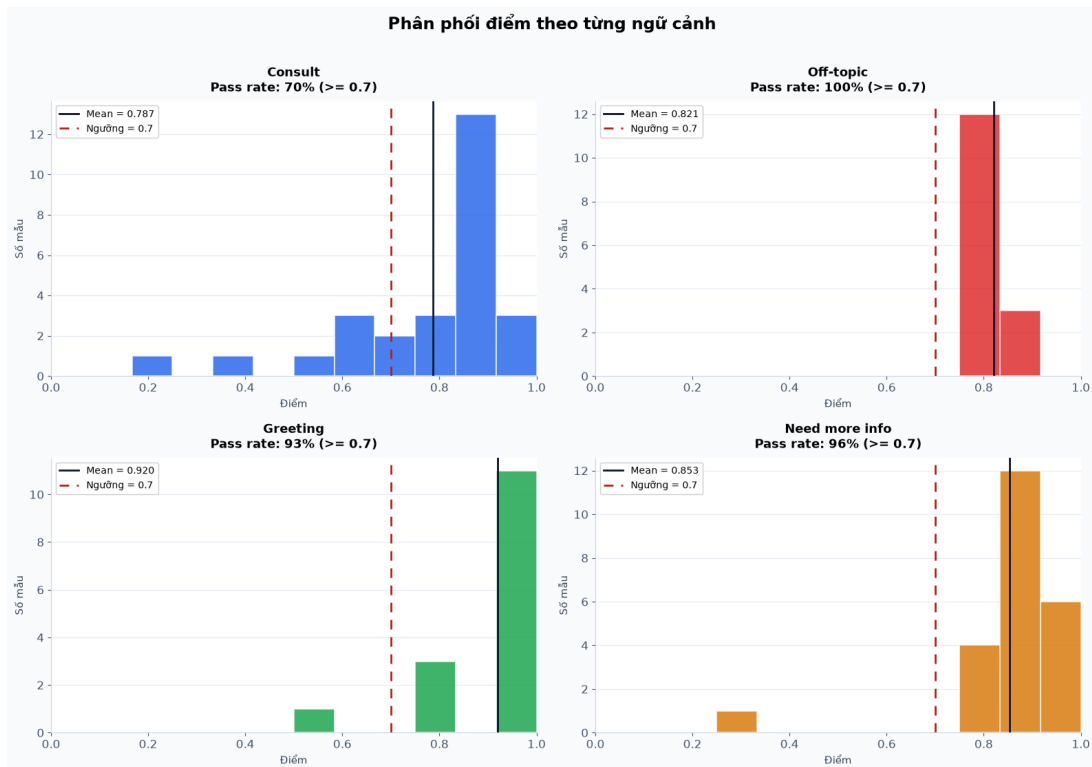
Luồng nghiệp vụ	Tên Metric	SL (n)	Trung bình	Độ lệch chuẩn	Min	Max
consult	Consultation_Quality	27	0.787	0.179	0.250	1.000
off-topic	OffTopic_Redirection	15	0.821	0.020	0.794	0.869
greeting	Chitchat_Greeting	15	0.920	0.136	0.527	1.000
need-more-info	Information_Gathering	23	0.853	0.130	0.315	0.995
TB có trọng số	—	80	0.837	—	—	—



Hình 3.8: Biểu đồ phân phối điểm số đánh giá theo luồng nghiệp vụ (Ngưỡng đạt: 0.7)

Phân tích số liệu thực nghiệm:

Dựa trên kết quả thu thập được, hệ thống Agentic RAG thể hiện hiệu năng tổng thể rất tích cực với điểm trung bình có trọng số đạt **0.837**, vượt xa ngưỡng tham chiếu an toàn (0.7). Hình 3.8 minh họa trực quan sự phân bố điểm số của từng luồng xử lý:



Hình 3.9: Điểm số đánh giá chi tiết theo từng tiêu chí GEval

Đi sâu vào phân tích từng luồng tương tác, đồ án rút ra các nhận định sau:

- **Luồng giao tiếp và định hướng (greeting, off-topic):** Đạt hiệu suất cao và ổn định nhất. Đáng chú ý, luồng off-topic có độ lệch chuẩn cực thấp (0.020), minh chứng cho khả năng nhận diện và điều hướng các câu hỏi lạc đề một cách đúng đắn.
- **Luồng thu thập thông tin (need-more-info):** Đạt trung bình **0.853** điểm. Hệ thống thực thi xuất sắc cơ chế "khóa" truy xuất để chủ động hỏi thêm các tiêu chí còn thiếu (slot-filling). Dù có điểm dị biệt (0.315) do LLM đôi khi sinh câu hỏi dài dòng, hiệu quả dẫn dắt tổng thể vẫn rất tích cực.
- **Luồng tư vấn chuyên sâu (consult):** Đạt **0.787** điểm. Đây là nhánh tác vụ phức tạp nhất do chất lượng đầu ra phụ thuộc trực tiếp vào hệ thống truy xuất tri thức (Retrieval). Việc mức điểm trung bình vượt qua ngưỡng tham chiếu an toàn đã minh chứng cho tính đúng đắn của chiến lược truy xuất lai (Hybrid Retrieval) kết hợp cùng cơ chế sinh phản hồi có cơ sở, giúp hệ thống vận hành hiệu quả và đáp ứng tốt các nghiệp vụ tư vấn khẩn khe.

3.3.6 Kết luận và Hướng phát triển

3.3.6.1 Kết quả đạt được

Hệ thống chatbot tư vấn mua bán xe dựa trên kiến trúc Agentic RAG đã đáp ứng tốt các mục tiêu ban đầu với những ưu điểm cốt lõi:

- **Kiểm soát tốt hiện tượng ảo giác (hallucination):** Hệ thống đảm bảo cung cấp thông số kỹ thuật và giá xe hoàn toàn dựa trên dữ liệu thực tế, giải quyết được bài toán về tính chính xác – yếu tố sống còn trong lĩnh vực thương mại ô tô.
- **Kiến trúc linh hoạt, dễ mở rộng:** Khung đồ thị LangGraph giúp hệ thống dễ dàng phân luồng xử lý nhiều tác vụ phức tạp (so sánh nhiều xe, thống kê dữ liệu, hỏi thông số) mà vẫn duy trì được sự ổn định của cấu trúc tổng thể.

3.3.6.2 Hạn chế còn tồn tại

Tuy nhiên, để đạt được độ chính xác cao, kiến trúc hiện tại phải chấp nhận sự đánh đổi (*trade-off*) về mặt hiệu năng:

- **Tốc độ phản hồi chậm (*High Latency*):** Thời gian chờ của người dùng bị kéo dài do luồng dữ liệu phải đi qua nhiều trạm xử lý (*nodes*) để suy luận và kiểm chứng.
- **Chi phí vận hành cao (*API Cost*):** Việc sử dụng LLM liên tục cho các bước định tuyến, kiểm tra chéo và tự đánh giá tiêu tốn lượng token lớn hơn đáng kể so với mô hình RAG tuyến tính thông thường.

3.3.6.3 Định hướng phát triển

Để hoàn thiện và đưa hệ thống vào triển khai thực tế, đề án đề xuất các hướng nâng cấp sau:

- **Khai thác thông tin và chuyển đổi khách hàng tiềm năng:** Mở rộng chức năng hệ thống để thu thập thông tin liên hệ của khách hàng (như số điện thoại, tên, email) khi đạt điều kiện tư vấn phù hợp. Các dữ liệu này, kết hợp cùng hồ sơ nhu cầu (*UserProfile*) đã được tóm tắt, sẽ tự động chuyển tiếp đến bộ phận bán hàng (*seller*) tương ứng nhằm tối ưu hóa cơ hội tiếp cận và chăm sóc khách hàng tiềm năng.
- **Chủ động dẫn dắt và định hướng nhu cầu:** Nâng cấp tác tử từ cơ chế phản hồi thụ động sang chủ động đặt câu hỏi định hướng. Chatbot sẽ biết cách khơi gợi nhu cầu, cô lập các lựa chọn tối ưu theo từng giai đoạn tương tác và từng bước dẫn dắt người dùng đi đến quyết định mua xe một cách tự nhiên.

3.4 Mô hình định giá xe

3.4.1 Giới thiệu

Bên cạnh hệ thống đề xuất và chatbot tư vấn, dự án xây dựng một **mô hình định giá xe** (price estimation) nhằm dự đoán giá thị trường của một chiếc xe từ thông số kỹ thuật và hình ảnh đăng bán. Mô hình được triển khai thành một dịch vụ độc lập và tích hợp vào trang Đăng bán: người bán nhập thông tin xe, hệ thống trả về mức giá thị trường ước tính kèm khoảng dao động, giúp định giá hợp lý ngay khi tạo tin đăng.

Quá trình xây dựng trải qua bốn giai đoạn chính — trích xuất và tiền xử lý dữ liệu, xây dựng đặc trưng (bao gồm đặc trưng thị giá), huấn luyện và so sánh nhiều hướng tiếp cận, và cuối cùng là đóng gói triển khai — được minh họa trong Hình 3.10.

3.4.2 Dữ liệu và tiền xử lý

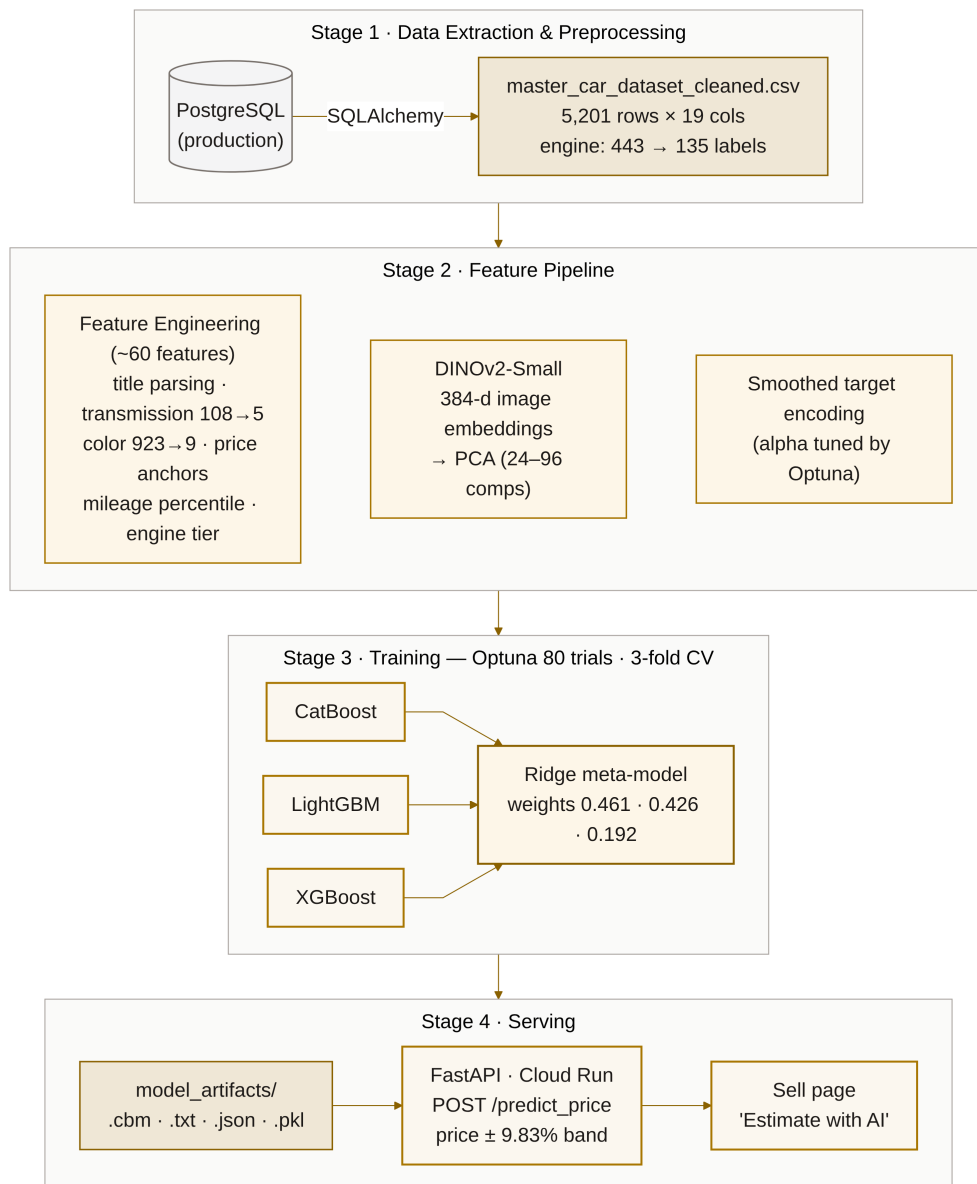
Dữ liệu huấn luyện được trích xuất trực tiếp từ cơ sở dữ liệu PostgreSQL của hệ thống bằng SQLAlchemy, join các bảng bài đăng, thông số xe và đánh giá thành một bảng phẳng gồm **5.201 dòng** × **19 cột** (hãng, dòng xe, năm, số dặm, nhiên liệu, hộp số, màu sắc, động cơ, 5 điểm đánh giá thành phần, tình trạng tai nạn/bảo hành, danh sách ảnh, và giá — biến mục tiêu).

Khâu làm sạch bao gồm loại bỏ bản ghi trùng lặp và thiếu trường quan trọng, lọc ngoại lai về giá/số dặm, và chuẩn hóa các trường boolean. Đáng chú ý nhất là trường *engine*: từ **443 giá trị** văn bản tự do (ví dụ chuỗi đặc tả dài của nhà sản xuất), một bộ phân tích dựa trên biểu thức chính quy trích xuất dung tích, cấu hình xy-lanh, thương hiệu động cơ (HEMI, EcoBoost, Duramax...), cờ tăng áp và loại truyền động, rút gọn còn khoảng **135 nhãn** nhất quán.

3.4.3 Xây dựng đặc trưng

Từ 19 cột thô, pipeline sinh ra khoảng **60 đặc trưng** mô hình hóa, trong đó các nhóm có đóng góp lớn nhất:

- **Phân tích tiêu đề:** bộ parser luật trích *model_name*, *trim*, hệ dẫn động (nhận diện từ token như xDrive, quattro, 4MATIC) và cờ xe off-road/EV từ tiêu đề tin đăng.
- **Chuẩn hóa phân loại:** hộp số từ 108 biến thể văn bản về 5 nhóm (CVT/Manual/DCT/Automatic/Other); màu ngoại thất từ 923 tên thương mại về 9 họ màu ngữ nghĩa.



Hình 3.10: Kiến trúc pipeline của mô hình định giá xe

- **Neo giá theo nhóm so sánh (price anchoring):** giá trung vị của các xe cùng nhóm (hãng × kiểu dáng × khoảng năm) được gắn vào từng dòng như một tín hiệu “giá thị trường tham chiếu”; luôn tính trên tập huấn luyện để tránh rò rỉ dữ liệu.
- **Bách phân vị số dặm trong nhóm tuổi:** thay vì chỉ dùng số dặm thô, mô hình biết xe chạy ít hay nhiều *so với các xe cùng tuổi*.
- **Đặc trưng thị giác:** mỗi ảnh tin đăng được mã hóa bằng **DINOv2-Small** (vision transformer tự giám sát của Meta AI) thành vector 384 chiều; nhiều ảnh của một xe được lấy trung bình. Vector này được giảm chiều bằng PCA (số thành phần 24–96, tinh chỉnh cùng các siêu tham số khác). Đặc trưng ảnh bắt được các tín hiệu bằng thông số không thể hiện: tình trạng xe, phong cách trim, chất lượng ảnh chụp.

Các cột phân loại có số lượng giá trị lớn (brand_model, trim, engine...) được mã hóa bằng **smoothed target encoding**: giá trị mã hóa là trung bình log-giá của nhóm, làm mượt về trung bình toàn cục theo số mẫu của nhóm (hệ số làm mượt α do Optuna tinh chỉnh trong khoảng 2–25), tránh việc mô hình tin tưởng thống kê tính trên quá ít mẫu.

3.4.4 Huấn luyện và so sánh các hướng tiếp cận

Ba mô hình cây quyết định (CatBoost, LightGBM, XGBoost) được huấn luyện với hàm mục tiêu hướng MAPE, trọng số mẫu tăng cường cho hai đuôi phân khúc giá (dưới \$20k và trên \$52k), và tìm kiếm siêu tham số bằng **Optuna** (80 trials, 3-fold cross-validation). Một **stacked ensemble** hợp nhất dự đoán out-of-fold của ba mô hình qua meta-model Ridge cũng được thử nghiệm, cùng một hướng tiếp cận hoàn toàn khác: fine-tune mô hình thị giác-ngôn ngữ **Qwen2-VL-7B** bằng QLoRA để dự đoán giá trực tiếp từ văn bản và ảnh.

Kết quả trên tập kiểm thử 20% (1.041 tin đăng) được tổng hợp trong Bảng 3.9.

Bảng 3.9: So sánh các hướng tiếp cận định giá trên tập kiểm thử

Mô hình / Hướng tiếp cận	MAPE	MAE
Baseline (LightGBM, không feature engineering)	14,94%	± \$5.801
CatBoost (tính chỉnh riêng)	10,10%	± \$3.816
XGBoost (tính chỉnh riêng)	9,81%	± \$3.523
Stacked Ensemble + Ridge	9,83%	± \$3.738
LightGBM (tính chỉnh riêng)	9,32%	± \$3.622
Qwen2-VL-7B (QLoRA)	7,38%	± \$2.728

Ba quan sát chính rút ra từ bảng so sánh:

- Feature engineering là nguồn cải thiện lớn nhất:** từ baseline 14,94% xuống 9,32% (giảm khoảng 38% tương đối) — lớn hơn nhiều so với khác biệt giữa các kiến trúc mô hình.
- Ensemble không vượt mô hình đơn tốt nhất:** do dự đoán của ba mô hình tương quan rất cao ($>0,9$), việc hợp nhất chỉ đạt 9,83%, không cải thiện so với LightGBM đơn lẻ; tuy nhiên ensemble cho sai số ổn định hơn giữa các phân khúc giá.
- Mô hình thị giác-ngôn ngữ chính xác nhất nhưng không khả thi về chi phí:** Qwen2-VL-7B đạt MAPE 7,38% — thấp nhất toàn dự án — nhưng yêu cầu GPU ≥ 24 GB VRAM chạy liên tục, vượt xa ngân sách vận hành, nên bị loại khỏi phương án triển khai.

3.4.5 Triển khai

Toàn bộ thành phần cần cho suy luận (ba mô hình cơ sở, meta-model Ridge, bộ PCA, bảng target encoding, thống kê neo giá, cấu hình) được serialize thành gói `model_artifacts/`. Dịch vụ định giá là một ứng dụng FastAPI đóng gói Docker, triển khai trên **Cloud Run**, nạp gói artifact khi khởi động và cung cấp endpoint `POST /predict_price`; ảnh người dùng tải lên được mã hóa DINOv2 ngay tại thời điểm suy luận. Dự đoán của ba mô hình cơ sở được hợp nhất qua meta-model và trả về kèm khoảng dao động $\pm 9,83\%$ tương ứng sai số kiểm thử.

Ở phía giao diện, trang Đăng bán tích hợp nút “*Estimate with AI*”: đọc thông tin xe người bán đã nhập, gọi dịch vụ định giá và hiển thị giá ước tính cùng khoảng giá, kèm tùy chọn điền trực tiếp mức giá gợi ý vào tin đăng.

4 KẾT QUẢ ĐẠT ĐƯỢC VÀ ĐÁNH GIÁ

4.1 Kết quả và Đánh giá

4.1.1 Kết quả xây dựng Giao diện hệ thống

Dự án đã phát triển thành công hệ thống website phục vụ nhu cầu tìm kiếm, so sánh và tư vấn mua bán xe ô tô với giao diện thân thiện, trực quan. Các tính năng chính đã được hiện thực hóa bao gồm:

Toàn bộ giao diện được xây dựng bằng React, TypeScript và Tailwind CSS, được triển khai công khai tại địa chỉ `carsalesfinder.com` và phục vụ bộ dữ liệu gồm **5.337 xe** (VIN duy nhất) được cập nhật tự động hàng tuần.

1. Giao diện Trang chủ và Tìm kiếm: Trang chủ (*Index Page*) cung cấp cái nhìn tổng quan về các mẫu xe nổi bật và các danh mục xe theo kiểu dáng/nhiên liệu. Trang Tìm kiếm (*Search Page*) cho phép người dùng lọc xe theo một bộ tiêu chí phong phú: hãng xe, khoảng giá, năm đăng ký (registration year), số dặm đã đi, loại nhiên liệu, hộp số, hệ dẫn động (drivetrain), kiểu dáng thân xe (body type), **màu ngoại thất** (được ánh xạ từ hàng trăm tên màu thô về 12 nhóm màu cơ bản và hiển thị dưới dạng swatch), và **danh sách tính năng** (features) với ngữ nghĩa AND — chỉ hiển thị các xe có đầy đủ tất cả tính năng được chọn.

2. Giao diện Chi tiết xe và So sánh: Trang Chi tiết xe (*Vehicle Detail Page*) hiển thị đầy đủ thông số kỹ thuật, hình ảnh, lịch sử xe (Vehicle History) và các bài đánh giá theo từng tiêu chí. Tính năng So sánh xe (*Compare Page*) cho phép người dùng đặt hai hoặc nhiều mẫu xe cạnh nhau để dễ dàng đối chiếu các thông số như động cơ, kích thước và giá bán.

3. Giao diện Chatbot Tư vấn: Giao diện Chat (*Chat Page*) được tích hợp trực tiếp vào hệ thống, đóng vai trò như một tư vấn viên ảo. Giao diện này hỗ trợ hiển thị tin nhắn văn bản (định dạng Markdown) kết hợp với các thẻ thông tin xe (Vehicle Cards) trực quan ngay trong khung chat, giúp người dùng dễ dàng xem nhanh thông tin xe được đề xuất. Người dùng đã đăng nhập còn có thanh bên lưu và khôi phục lại các phiên hội thoại trước đó.

4. Giao diện Đăng bán và Định giá bằng AI: Trang Đăng bán (*Sell Page*) cho phép người bán nhập thông tin xe và tích hợp một nút **gợi ý giá bằng AI**: từ các thông tin đã nhập, hệ thống gọi mô hình định giá (được trình bày ở phần sau) để đưa ra mức giá thị trường ước tính kèm khoảng dao động, giúp người bán định giá hợp lý.

4.1.2 Đánh giá Hệ thống Chatbot (Qualitative Evaluation)

Do đặc thù của hệ thống Chatbot tư vấn phụ thuộc nhiều vào ngữ cảnh và nhu cầu cá nhân hóa, nhóm tiến hành đánh giá định tính (Qualitative) thông qua các kịch bản sử dụng thực tế. Chatbot sử dụng kiến trúc Agentic LangGraph đã thể hiện khả năng xử lý linh hoạt qua các kịch bản sau:

1. Kịch bản tư vấn từ nhu cầu mơ hồ (Slot-filling):

- **Tình huống:** Người dùng chỉ cung cấp thông tin "I want a reliable family SUV under \$30,000" (bộ dữ liệu là thị trường Mỹ, đơn vị USD).
- **Kết quả xử lý:** Thay vì đưa ra một danh sách dài, Chatbot tự động nhận diện ý định và cập nhật *UserProfile* (budget_max: 30000, body_type: SUV). Hệ thống nhận thấy còn thiếu thông tin quan trọng theo các core slot (ví dụ: loại nhiên liệu) nên đã chủ động đặt câu hỏi dẫn dắt: "Do you have a preference for the fuel type — gasoline, hybrid, or electric?".
- **Đánh giá:** Khả năng duy trì ngữ cảnh (multi-turn) và thu thập thông tin (slot-filling) hoạt động tự nhiên, giúp thu hẹp phạm vi tìm kiếm hiệu quả mà không gây quá tải thông tin cho người dùng.

2. Kịch bản so sánh xe phức tạp:

- **Tình huống:** "So sánh giúp tôi Honda CR-V và Mazda CX-5".

- **Kết quả xử lý:** Bộ định tuyến (Intent Router) điều hướng chính xác vào luồng `compare_retrieve`. Chatbot tổng hợp thông số của cả hai dòng xe từ cơ sở dữ liệu và sinh ra câu trả lời so sánh đối chiếu về giá, hiệu suất động cơ, và không gian nội thất, đồng thời đính kèm hai thẻ xe để người dùng click xem chi tiết.
- **Đánh giá:** Hệ thống giảm tải thời gian tra cứu thủ công cho người dùng, cung cấp câu trả lời khách quan dựa trên dữ liệu thực tế (Grounded Generation).

4.1.3 Khả năng truy xuất dữ liệu (Hybrid Retrieval)

Hệ thống sử dụng cơ chế Hybrid Retrieval kết hợp giữa SQL Hard-filter và Qdrant Semantic Search.

- **Tốc độ phản hồi (Latency):** Việc áp dụng SQL để lọc cứng các siêu dữ liệu (Metadata) như giá tiền, hãng xe, năm sản xuất trước khi đưa vào không gian vector (Qdrant) giúp giảm thiểu đáng kể số lượng vector cần tính toán. Nhờ đó, hệ thống đạt tốc độ phản hồi truy vấn rất nhanh, đáp ứng tốt trải nghiệm trò chuyện liên tục của người dùng.
- **Độ chính xác ngữ nghĩa:** Qdrant đảm nhiệm việc tìm kiếm ngữ nghĩa đối với các yêu cầu không cấu trúc (ví dụ: "xe có cảm giác lái thể thao, cách âm tốt"), mang lại các kết quả đề xuất vượt ra khỏi các bộ lọc từ khóa truyền thống. Qua thử nghiệm thực tế, hệ thống luôn trả về top các kết quả thỏa mãn đồng thời cả ràng buộc cứng (giá, hãng) và ràng buộc mềm (phong cách, cảm giác lái).

4.1.4 Mô hình định giá xe (Price Estimation)

Bên cạnh hệ thống đề xuất và chatbot, dự án hiện thực hóa một mô hình học máy độc lập để **dự đoán giá thị trường** của xe. Mô hình nhận đầu vào là các thông tin đặc tả (hãng, dòng xe, năm, số dặm, nhiên liệu, hộp số, màu sắc) kết hợp với đặc trưng trích xuất từ ảnh xe bằng mô hình thị giác **DINOv2**. Kết quả dự đoán được hợp nhất từ nhiều mô hình cây quyết định (CatBoost, LightGBM, XGBoost) qua một meta-model, đạt sai số MAPE khoảng **9,83%** trên tập kiểm thử.

Mô hình được đóng gói thành một dịch vụ riêng trên Cloud Run và được tích hợp vào trang Đăng bán: khi người bán nhập thông tin xe, hệ thống trả về mức giá ước tính kèm khoảng dao động (ví dụ: một chiếc Toyota Camry 2020, 35.000 dặm được ước tính khoảng \$28.200, khoảng \$25.400 – \$31.000).

4.1.5 Tổng kết

Hệ thống tư vấn mua bán xe đã hoàn thiện với khả năng hoạt động ổn định. Giao diện được thiết kế hiện đại, đáp ứng đầy đủ luồng người dùng từ tìm kiếm, so sánh đến tư vấn trực tiếp qua AI. Đặc biệt, luồng Chatbot Agentic đã chứng minh được tính hiệu quả trong việc chủ động thu thập thông tin, cá nhân hóa đề xuất và phản hồi nhanh chóng, giải quyết tốt bài toán tư vấn mua bán xe vốn đòi hỏi tổng hợp nhiều thông tin đa chiều.

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết Luận

Car Recommendation System là một hệ thống tích hợp hoàn chỉnh, kết hợp **Data Engineering, Machine Learning, Natural Language Processing** và **modern web technologies** để giải quyết bài toán tìm kiếm và tư vấn mua xe thông minh.

Những đóng góp kỹ thuật nổi bật của dự án:

- **Pipeline dữ liệu tự động hoàn toàn:** Triển khai pipeline thu thập và xử lý dữ liệu hàng tuần sử dụng **Temporal** (orchestration) với kiến trúc fail-stop 3-workflow: WeeklyCrawl → Transform → ML. Dữ liệu trải qua **Medallion Architecture** (Bronze → Silver → Gold) được quản lý bởi **dbt**, đảm bảo tính incremental và idempotent.
- **Hệ thống đề xuất 3-stage hybrid:** Thiết kế Recommendation Engine theo pipeline **Candidate Generation** → **Ranking** → **Re-ranking**. Stage 1 gồm 4 recaller song song (Collaborative, Content, Vector, Popularity); Stage 2 dùng **WeightedLinearRanker** với 7 features có trọng số cấu hình qua YAML; Stage 3 dùng **MMR-Reranker** (Maximal Marginal Relevance) để đảm bảo đa dạng kết quả với hard caps theo brand/model.
- **Chatbot tư vấn Agentic RAG:** Xây dựng chatbot trên **LangGraph StateGraph** với 7 loại intent routing (compare, analytics, specs, specific, vague, chitchat, off_topic), cơ chế **slot-fill profile** và **hybrid retrieval** (SQL hard-filter + Qdrant semantic search trên collection car_vectorize).
- **Kiến trúc database Medallion:** Thiết kế multi-schema (Bronze/Silver/Gold) với Silver theo mô hình chiều (dimensional model) do dbt quản lý, và Gold gồm các bảng denormalized (marts) phục vụ frontend cùng bảng user-domain do ứng dụng ghi trực tiếp.
- **Mô hình định giá xe (Price Estimation):** Bổ sung một mô hình học máy độc lập dự đoán giá thị trường của xe từ thông tin đặc tả (hãng, dòng xe, năm, số dặm, nhiên liệu, hộp số) kết hợp đặc trưng hình ảnh (DINOv2). Mô hình được triển khai thành dịch vụ riêng và tích hợp vào trang đăng bán để gợi ý mức giá hợp lý cho người bán.
- **Triển khai sản xuất trên GCP:** Hệ thống được đóng gói bằng Docker và triển khai lên **Google Cloud Platform**: Cloud Run cho backend và frontend, AlloyDB (PostgreSQL 17) cho cơ sở dữ liệu, Qdrant Cloud cho vector database, và **Temporal tự triển khai (self-hosted)** trên máy ảo GCE. Hệ thống được truy cập công khai tại carsalesfinder.com.

5.2 Ứng Dụng Thực Tế

Hệ thống có thể được triển khai trong các tình huống thực tế sau:

1. **Nền tảng thương mại điện tử** cho showroom và đại lý xe: cung cấp trải nghiệm mua sắm cá nhân hóa với chatbot AI tư vấn 24/7, hỗ trợ so sánh xe và gợi ý thông minh theo hành vi người dùng.
2. **Sàn giao dịch xe trực tuyến:** giúp người mua tìm xe phù hợp nhanh chóng thông qua tìm kiếm bằng ngôn ngữ tự nhiên thay vì lọc theo biểu mẫu phức tạp, đồng thời phân tích thống kê kho xe theo thời gian thực.
3. **Nền tảng phân tích khách hàng** cho nhà sản xuất và đại lý: khai thác thông tin từ dữ liệu hành vi người dùng (lượt xem, nhấp chuột, lưu yêu thích, liên hệ) để tối ưu kho xe và chiến lược định giá.
4. **Kiến trúc có khả năng mở rộng:** Nền tảng pipeline Temporal + dbt + vector database có thể được điều chỉnh cho các lĩnh vực dọc (vertical) khác như bất động sản, điện tử tiêu dùng, hoặc bất kỳ lĩnh vực nào cần tìm kiếm và gợi ý cá nhân hóa trên dữ liệu được thu thập định kỳ.

5.3 Hướng Phát Triển

Bên cạnh những kết quả đã đạt được, hệ thống vẫn còn một số hướng có thể tiếp tục hoàn thiện trong tương lai:

- **Lưu trữ hồ sơ người dùng (UserProfile) bền vững:** hiện hồ sơ hội thoại của chatbot được lưu in-memory theo tiến trình; việc ghi hồ sơ xuống cơ sở dữ liệu sẽ cho phép triển khai đa instance và khôi phục ngữ cảnh khi backend khởi động lại.
- **Nâng cấp truy xuất cho chatbot:** thử nghiệm các chiến lược re-ranking (cross-encoder) hoặc Reciprocal Rank Fusion để hợp nhất kết quả SQL và vector một cách có trọng số, thay cho việc ghép ngữ cảnh đơn giản hiện tại.
- **Huấn luyện ranker cho hệ thống đề xuất:** thay bộ WeightedLinearRanker tuyến tính bằng mô hình learning-to-rank (ví dụ LightGBM ranker) khi lượng dữ liệu tương tác đủ lớn, đồng thời bổ sung đánh giá offline (Precision@K, NDCG, Diversity).
- **Tối ưu chi phí mô hình định giá:** khảo sát các kỹ thuật lượng tử hóa/chưng cất (distillation) để đưa mô hình thị giác-ngôn ngữ (đạt MAPE thấp nhất) về mức chi phí vận hành khả thi, thu hẹp khoảng cách với phương án cây quyết định đang triển khai.
- **Mở rộng nguồn dữ liệu:** bổ sung thêm nguồn tin đăng ngoài Cars.com và cập nhật tần suất crawl để dữ liệu phản ánh thị trường sát thời gian thực hơn.